

第 19 章: Function Odds and Ends (函数的边角细节)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本章是"第四部分: 函数"的收官章节, 把前几章没讲完的函数主题一次性补齐: 函数设计原则、递归函数、函数对象(一等对象模型、属性、注解、装饰器预告)、lambda 表达式, 以及 map/filter/reduce 三个函数式编程工具。读完本章, 你将真正理解"函数在 Python 中也是数据"这句话, 并掌握把函数作为一等公民(first-class citizen)组合使用的全部基础——这是下一章生成器(generator)与推导式(comprehension)深入的前奏。

19.1 章节引言 (Chapter Intro)

英文原文

This chapter presents a medley of function-related topics: recursive functions; function attributes, annotations, and decorations; and more on both the lambda expression and functional-programming tools such as map and filter. These are all somewhat advanced tools that, depending on your job description, you may

not encounter on a regular basis. Because of their roles in some domains, though, a basic understanding can be useful. `lambda`, for instance, makes regular appearances in GUIs, and functional programming techniques have grown common in Python code. Some of the art of using functions lies in the interfaces between them, so we will also explore some general function design principles here. The next chapter continues the advanced themes here with an exploration of generator functions and expressions and a revival of list comprehensions in the context of the functional tools we will study here.

中文翻译

本章呈现一组与函数相关的杂锦 (medley) 话题：递归函数 (recursive functions)；函数属性 (attributes)、注解 (annotations) 和装饰 (decorations)；以及对 `lambda` 表达式和 `map`、`filter` 等函数式编程工具的更多探讨。这些都是多少有些高级的工具，取决于你的工作性质，你可能不会经常遇到它们。但正因为它们在某些领域的地位，对它们有一个基本的了解会很有用。例如，`lambda` 在 GUI 中频繁出场，函数式编程技术也已在 Python 代码中变得常见。使用函数的一部分艺术在于它们之间的接口 (interfaces)，因此我们也会在这里探讨一些通用的函数设计原则。下一章将继续本章的高级主题——探索生成器函数 (generator functions) 与生成器表达式 (generator expressions)，并在我们这里学到的函数式工具背景下重新审视列表推导式 (list comprehensions)。

深度理解

·核心概念：本章标题 "Odds and Ends"（零碎杂物、边角料）非常贴切——这一章就是函数主题的"收尾工作"，把散落的碎片拼齐。作者明确说这些工具"偏高级、不一定天天用"，但它们在特定领域（GUI 回调、函数式编程）价值巨大，值得了解。

·定位关系：注意作者把下一章（第 20 章生成器）与本章串成一条线——map 返回惰性迭代器、列表推导式加括号变生成器表达式，这些"按需生成"的思想正是下一章的核心。本章是下一章的"地基"。

·为什么这样设计：Lutz 的书一直遵循"由浅入深、分章消化"的节奏。函数的主体（定义、参数、作用域）已在第 16~18 章讲完，剩下这些"相关性高但不成体系"的内容集中放一章，便于读者在进入生成器前有完整的函数观。

·实际问题：工作中你会遇到：界面回调要写小函数（lambda）、处理嵌套 JSON/目录树要递归、给函数挂附加信息要属性/注解、批量处理序列要 map/filter/reduce。本章就是这些场景的"解决方案手册"。

·初学者误区：很多初学者以为"函数"这个概念在学到 def 之后就学完了。实际上 def 只是函数世界的入口——函数是对象、可传递、可内省、可附注，这些"高级"能力才是 Python 函数真正强大的地方。

19.2 函数设计概念 (Function Design Concepts)

英文原文

Now that we've studied function essentials in Python, let's open this chapter with some perspective. When you start using functions in earnest, you're faced with choices about how to glue components together—for instance, how to decompose a task into purposeful functions (known as cohesion), and how your functions should communicate (called coupling). You also need to take note of the size of your functions because it directly impacts code usability. Some of this falls into the category of structured analysis and design, but it applies to Python code as to any other. We explored some ideas related to function and module coupling in Chapter 17 when studying scopes, but here is a review of a few general guidelines for readers new to function design principles:

- Coupling:** use arguments for inputs and return for outputs. Generally, you should strive to make a function independent of the world outside of it. Arguments and return statements are often the best ways to isolate external dependencies to a small number of well-known places in your code.
- Coupling:** use global variables only when truly necessary. As we've seen, global variables (i.e., names in the enclosing module) are usually a poor way for fu

ctions to communicate. They can create dependencies and timing issues that make programs difficult to debug, change, and reuse. Coupling: don't change mutable arguments unless the caller expects it. As we've also seen, functions can change parts of passed-in mutable objects, but as with global variables, this creates a tight coupling between the caller and callee, which can make a function too specific and brittle. Cohesion: each function should have a single, unified purpose. When designed well, each of your functions should do one thing—something you can summarize in a simple declarative sentence. If that sentence is very broad (e.g., "this function implements my whole program") or contains lots of conjunctions (e.g., "this function gives employee raises and submits a pizza order"), you might want to think about splitting it into separate and simpler functions. Otherwise, there is no way to reuse the code of the individual steps embedded in the function. Size: each function should be relatively small. This naturally follows from the preceding goal, but if your functions start spanning multiple pages on your display, it's probably time to split them. Especially given that Python code is so concise to begin with, a long or deeply nested function is often a symptom of design problems. Keep it simple, and keep it short. Coupling: avoid changing variables in another module file directly. We also introduced this concept in Chapter 17, and we'll revisit it in the next part

of the book when we focus on modules. For reference, though, remember that changing variables across file boundaries sets up a coupling between modules similar to how global variables couple functions—the modules become difficult to understand and reuse separately. Use accessor functions whenever possible, instead of direct assignment statements. Figure 19-1 summarizes the ways functions can talk to the outside world; inputs may come from items on the left side, and results may be sent out in any of the forms on the right. Nonlocals might belong in this sketch too, but they're mostly a state-retention tool in the same category as other local variables. Despite the array of options, good function designs prefer to use only arguments for inputs and return statements for outputs, whenever possible. Figure 19-1. Function execution environment

Of course, there are plenty of exceptions to the preceding design rules, including some related to Python's OOP support. As you'll see in Part VI, Python classes depend on changing a passed-in mutable object—class functions set attributes of an automatically passed-in argument called `self` to change per-object state information (e.g., `self.edition=6`). Moreover, if classes are not used, global variables are a straightforward way for functions in modules to retain single-copy state between calls. Side effects are usually dangerous only if they're unexpected. In general though, you should strive to minimize external dependen

cies in functions and other program components. The more self-contained a function is, the easier it will be to understand, reuse, and modify. Making code as freestanding as possible is especially important when functions go multilevel with recursion, per the next section.

中文翻译

既然我们已经学完了 Python 中函数的基本知识，让我们带一点全局视角来开启本章。当你认真开始使用函数时，你会面临“如何把组件粘在一起”的选择——例如，如何把一个任务分解成有意义的小函数（这称为内聚（cohesion）），以及你的函数之间应该如何通信（这称为耦合（coupling））。你还需要留意函数的大小（size），因为它直接影响代码的可用性。其中一部分内容属于“结构化分析与设计”的范畴，但它和适用于其他语言的规则一样适用于 Python 代码。在第 17 章研究作用域时，我们已经探讨过一些与函数和模块耦合相关的思想，但这里还是为刚接触函数设计原则的读者复习几条通用准则：耦合：用参数（arguments）做输入，用 return 做输出。一般来说，你应该努力让函数独立于外部世界。参数和 return 语句往往是把外部依赖隔离到代码中少数几个众所周知的点的最好方式。耦合：只在真正必要时使用全局变量（global variables）。如我们看到的，全局变量（即所在模块中的名字）通常是函数间通信的糟糕方式。它们会制造依赖关系和时序问题，使程序难以调试、修改和复用。耦合：不要修改可变参数（mutable arguments），除非调用者期望如此。我们也看到过，函数可以修改传入的可变对象的部分内容，但和全局变量一样

，这会在调用者和被调用者之间制造紧密耦合，使函数过于特殊化而变得脆弱。内聚：每个函数应该只有一个、统一的目的。设计良好时，你的每个函数应该只做一件事——一件你能用一句简单的陈述句概括的事。如果那句话非常宽泛（例如“这个函数实现了我整个程序”），或者包含很多连词（例如“这个函数给员工加薪并且提交一份披萨订单”），你可能就该考虑把它拆成多个更简单独立的函数了。否则，函数内部单个步骤的代码将无法被复用。大小：每个函数应该相对较小。这自然承接上一条目标，但如果你发现函数开始在你的屏幕上跨越多页，多半是时候拆分它了。尤其考虑到 Python 代码本身已经很简洁，一个冗长或深嵌套的函数往往是设计问题的征兆。保持简单，保持简短。耦合：避免直接修改另一个模块文件中的变量。我们在第 17 章引入过这个概念，并将在本书下一部分聚焦模块时再次讨论。不过先记住：跨越文件边界修改变量，会在模块之间建立起一种类似于全局变量耦合函数之间的耦合关系——这些模块会变得难以独立理解和复用。尽可能使用访问器函数（accessor functions），而不是直接赋值语句。图 19-1 总结了函数与外界“对话”的方式；输入可以来自左侧的各种形式，结果可以以右侧的任何一种形式送出。Nonlocal 也许也应该画进这张图，但它们主要是一种状态保留工具，与其它局部变量属于同一类。尽管选项繁多，良好的函数设计仍倾向于：只要可能，只用参数做输入、只用 return 语句做输出。图 19-1. 函数执行环境当然，上述设计规则有很多例外，其中包括一些与 Python 面向对象（OOP）支持相关的例外。正如你将在第六部分看到的，Python 的类依赖于修改传入的可变对象——类中的函数通过设置一个自动传入的参数 `self` 的属性来改变每个对象的状态信息（例如 `self.edition =`

6)。此外，如果不使用类，全局变量是模块内函数在调用之间保留单份状态（single-copy state）的一种直接方式。副作用（side effects）通常只有在出乎意料时才是危险的。不过总的来说，你应该努力将函数及其它程序组件的外部依赖降到最低。一个函数越是自包含（self-contained），就越容易理解、复用和修改。尽可能让代码“自立门户”，这一点在函数像下一节那样进入多级递归时尤为重要。

深度理解

·核心概念：本节提出了软件工程的两个基本度量——内聚（一个模块/函数内部元素的相关程度）和耦合（模块之间的关联程度）。设计口诀可以浓缩为：高内聚、低耦合。函数应当“只做一件事，并且把这件事做好”，与外界的通信尽量走参数和返回值。

·为什么这样设计：耦合度越高，一处改动波及面越大。想象搭积木：每块积木如果都跟旁边的积木长在一起（强耦合），想换掉其中一块就得拆掉整座塔；如果每块积木独立（弱耦合），随时可以替换任意一块。函数的可复用性、可测试性本质上都取决于耦合度。

·底层视角：作者提到“side effects are usually dangerous only if they're unexpected”（副作用通常只有在出乎意料时才是危险的）——Python 中列表、字典等可变对象按引用传递，函数内 `L.append(...)` 会直接改到调用者的对象。这是 Python 的设计取舍：效率高、灵活，但容易踩坑。类的 `self` 属性赋值就是“利用副作用”的典型正面案例。

·实际问题：面试和代码审查中，"这个函数做了几件事？"
"这个模块为什么改不动？"——答案往往就是内聚太低或耦合太高。把大函数拆小、用参数代替全局变量、用返回值代替副作用，是新手到高手的分水岭。

·初学者误区：初学者最容易犯的错是"用全局变量省事"。全局变量看似方便，但函数一旦读写全局名字，这个函数就跟模块的全局命名空间绑死了：测试时难以隔离、并发时容易出时序问题、改名时牵连一片。记住 Lutz 的话："只在真正必要时使用"。

19.3 递归函数 (Recursive Functions)

英文原文

We mentioned recursion in relation to comparisons of core types in Chapter 9. While discussing scope rules near the start of Chapter 17, we also briefly noted that Python supports recursive functions—functions that call themselves either directly or indirectly in order to loop. In this section, we'll explore what this looks like in our functions' code. Recursion is a somewhat advanced topic, and it's relatively uncommon to see in Python, partly because Python's procedural shed includes simpler looping tools. Still, it's a useful technique to know about, as it allows programs to traverse structures that have arbitrary and unpredictable shapes and depths—planning travel routes, analyzing language, and crawling links on the web, for example. R

recursion is even an alternative to simple loops and iterations, though not necessarily the simplest or most efficient one.

中文翻译

我们在第 9 章讨论核心类型比较时提到过递归。在第 17 章开头讨论作用域规则时，我们也简单提到过 Python 支持递归函数 (recursive functions) ——直接或间接调用自身以构成循环的函数。在本节中，我们将看看它在函数代码中是什么样子。递归是一个有点高级的话题，而且在 Python 中相对少见，部分原因是 Python 的过程式工具箱里有更简单的循环工具。尽管如此，它仍是一项值得了解的技巧，因为它让程序能够遍历任意、不可预测的形状和深度的结构——例如规划旅行路线、分析语言、爬取网页链接。递归甚至可以作为简单循环和迭代的替代方案，尽管它不一定是其中最简单或最高效的。

深度理解

- 核心概念：递归 = 函数调用自己。它把"一个大问题"拆成"更小的同类问题"（递归步）直到不可再拆（递归终止条件）。分而治之 (divide and conquer) 是它的灵魂。
- 底层实现：Python 在每次函数调用时都会在运行时调用栈 (call stack) 上压入一个栈帧 (frame)，保存局部变量、返回地址等信息。递归调用就是在栈上不断压入新的帧，返回时逐个弹出——所以每个递归层级的局部变量互不干扰。
- 设计原因：递归的存在是因为某些结构的深度未知（任意

嵌套的目录树、语法树、JSON)。循环需要你预先写死"多少层"，而递归把"层数"交给运行时自己决定。

·实际问题：Web 爬虫的链接图、文件系统遍历、XML/JSON 解析、回溯算法（八皇后、排列组合）都靠递归。但线性求和这种场景用递归就是"杀鸡用牛刀"。

·初学者误区：① 忘记写终止条件 → RecursionError；② 以为递归和循环完全等价——循环适合线性结构，递归适合嵌套结构；③ 没意识到每个递归层级都占用独立的内存栈帧，深递归可能爆栈。

代码分析

```
def mysum(L):  
    if not L:  
        return 0  
    else:  
        return L[0] + mysum(L[1:])    #
```

解释器如何处理：调用 mysum([1, 2, 3, 4, 5]) 时：

1. 第 1 层帧：L = [1,2,3,4,5]，计算 1 + mysum([2,3,4,5])——求值右侧表达式时又发起一次调用，压入第 2 层帧。
2. 第 2 层帧：L = [2,3,4,5]，计算 2 + mysum([3,4,5])...
...如此层层压栈，直到 L = [] 时 not L 为真，返回 0。
3. 栈开始回卷 (unwind)：0 返回给上一层，5 + 0 → 5，再返回给上一层 4 + 5 → 9.....最终回到最外层得到 15。
4. 关键点：每一层帧里的 L 是各自独立的局部变量——L[

1:] 创建了新的切片列表，所以各层 "记住" 了自己那一份列表片段。这就是作者说的 "each remembers its own segment of the list"。

设计思想：递归的两个要素——递推关系 ($\text{sum}(L) = L[0] + \text{sum}(L[1:])$) 和终止条件 (not L 时返回 0)。写递归就是同时想清楚这两件事，缺一不可。

19.3.1 用递归求和 (Summation with Recursion)

英文原文

Let's turn to some examples. To sum a list (or other sequence) of numbers, we can either use the built-in sum function or write a more custom version of our own. Example 19-1 shows what a custom summing function might look like when coded with recursion.

Example 19-1. mysum.py

```
def mysum(L): if not L: return 0 else: return L[0] + mysum(L[1:]) # Call myself recursively
```

To use, either add self-test code to the bottom of this file and run it as a script, or import it as a module and test at the REPL (again, the file may need to be in the folder where you're working either way, per Chapter 3). With the latter:

```
>> from mysum import mysum # Import file as a module in a REPL
>> mysum([1, 2, 3, 4, 5]) # Sum all the
```

numbers in any sequence 15

At each level, this function calls itself recursively to compute the sum of the rest of the list, which is later added to the item at the front. This recursive loop ends and zero is returned when the list becomes empty. When using recursion like this, each open level of call to the function has its own copy of the function's local scope on the runtime call stack. Here, that means `L` is different in each level, so each remembers its own segment of the list.

If this is difficult to understand (and it often is for new programmers), try adding a print of `L` to the function and run it again, to trace the current list at each call level; here's the required mod pasted at the REPL for variety:

```
>> def mysum(L):  
    print(L) # Trace recursive levels if not L: # L shorter at  
    each level return 0 else: return L[0] + mysum(L[1:]) >  
> mysum([1, 2, 3, 4, 5]) [1, 2, 3, 4, 5] [2, 3, 4, 5] [3, 4, 5]  
[4, 5] [5] [] 15
```

As you can see, the list to be summed grows smaller at each recursive level, until it becomes empty—the termination of the recursive loop. The sum is then computed as the recursive calls unwind on returns.

中文翻译

让我们来看一些例子。要对于一个数字列表（或其他序列）求和，我们可以用内置的 `sum` 函数，也可以自己写一个更定制化的版本。示例 19-1 展示了用递归编码的自定义求和函数会是什么样。示例 19-1. `mysum.py`

```
python def mysum(L): if not L: return 0 else: return L[0] + mysum(L[1:])
```

递归调用我自己使用方式有两种：要么在文件底部添加自测（self-test）代码并作为脚本运行，要么把它作为模块导入后在 REPL 里测试（同样地，无论哪种方式，文件可能都需要放在你当前工作的文件夹里，参见第 3 章）。以后一种方式为例：

```
python >>> from mysum import mysum
```

在 REPL 中把文件作为模块导入

```
>>> mysum([1, 2, 3, 4, 5])
```

对任意序列中的所有数字求和 15 在每一层，这个函数都递归调用自身来计算列表剩余部分的和，之后再与最前面的那一项相加。当列表变空时，这个递归循环终止并返回 0。像这样使用递归时，函数每一次未返回的调用，在运行时调用栈（runtime call stack）上都有自己的局部作用域副本。这里就意味着每一层里的 `L` 都不同，所以每一层都记住了列表属于自己的那一段。如果这很难理解（新程序员常常如此），试试在函数里加一条 `print(L)` 再运行一次，追踪每个调用层级当前的列表；下面是在 REPL 中粘贴所需修改后的样子：

```
python >>> def mysum(L): print(L) # 追踪递归层级
if not L: # L 每层都在变短
return 0 else: return L[0] + mysum(L[1:])
```

```
>>> mysum([1, 2, 3, 4, 5])
```

[1, 2, 3, 4, 5] [2, 3, 4, 5] [3, 4, 5] [4, 5] [5] [] 15 如你所见，要求和的列表在每个递归层级都变得更小，直到变为空——这就是递归循环的终止点。随后，和是在递归调用返回时回卷

(unwind) 的过程中计算出来的。

深度理解

- 核心概念：mysum 展示了递归的经典模板——“处理第一项 + 递归处理剩余项”。求和公式 $\text{sum}(L) = L[0] + \text{sum}(L[1:])$ 是递推关系，if not L 是基准情形 (base case)。
- 底层实现：每个递归层级 = 调用栈上一个独立栈帧。作者特意强调 “L is different in each level”—— $L[1:]$ 会产生新列表，因此各层引用的是不同的切片对象，各层的局部变量互不干扰。这也是递归“安全”的机制：栈帧天然隔离。
- 为什么这样设计：把“对整体求和”翻译成“对头部求和、对尾部递归”，是函数式编程 (F#、Haskell、Lisp) 处理列表的标准姿态——列表被定义成“头 + 尾”。Python 从这类语言中继承了递归的用法，但日常更偏爱循环。
- 实际问题：print(L) 追踪法是调试递归的利器——把“看不见的调用层级”变成“看得见的逐步输出”。任何递归难懂时，第一步就是打印各层的输入。
- 初学者误区：① 以为递归会在到达终止条件后“瞬间得出答案”——实际上答案是在返回途中 (unwind) 逐步拼出来的；② 忽略切片 $L[1:]$ 每次创建新列表的内存开销——递归简单但未必高效。

19.3.2 编码替代方案 (Coding Alternatives)

英文原文

Interestingly, we can use Python's if/else ternary expression (described in Chapter 12) to save some code real estate here. We can also generalize for any summable type (which is easier if we assume at least one item in the input) and use extended-unpacking assignment to make the first/rest unpacking simpler (as covered in Chapter 11). Example 19-2 collects all three of these mods, ready to be run in a file or pasted into a REPL. Example 19-2. mysumalts.py

```
def mysum(L): return 0 if not L else L[0] + mysum(L[1:]) # Use ternary expression
def mysum(L): return L[0] if len(L) == 1 else L[0] + mysum(L[1:]) # Any type, assume one+
def mysum(L): first, rest = L
```

```
    return first if not rest else first + mysum(rest) # Use extended unpacking
```

When tested individually, all three of these alternatives handle numeric summation the same as the original:

```
>>> mysum([1, 2, 3, 4, 5]) 15
```

Uniquely, the latter two fail for empties (e.g., `mysum([])`), but handle sequences of any object type that supports `+`, not just numbers (for strings, the effect is similar to `"".join(L)`):

```
>>> mysum(('h','a','c','k')) # The last two fail on mysum([])'hack'
```

```
>>> mysum(['hack','app','code']) # But they support nonnumeric types'hackappcode'
```

Run some tests on your own for more insight. If you study these three variants, you'll also find that: - The latter two work on a single string argument (e.g., `mysum('hack')`), because strings are sequences of one-character strings (though this use case isn't very useful: you get back the same string). - The third variant also works on arbitrary iterables, including open input files (`mysum(open(name))`), but the others' indexing generally fails on nonsequences (see Chapter 14 for extended-unpacking demos). You may also notice that the third variant's unpacking assignment is similar to a collector in a function header, and it's tempting to re-code it as such. This won't quite work, though, because it would expect individual arguments, not a single iterable—unless we also star both the top-level input and recursive call. Here's the end result, though by summing discrete arguments, it solves a different problem than both the prior versions and built-in `sum`:

```
>>> def mysum(first, rest): return first if not rest else first + mysum(rest) >>> mysum([1, 2, 3, 4, 5]) 15
```

```
>>> mysum('hack')'hack'
```

Finally, bear in mind that recursion can be either direct, as in the examples so far, or indirect, as in the foll

owing—a function that calls another function, which calls back to its caller. The net effect is the same, though there are two function calls at each level instead of one:

```
>>> def mysum(L): if not L: return 0 return nonempty(L) # Call a function that calls me
>>> def nonempty(L): return L[0] + mysum(L[1:]) # Indirectly recursive
>>> mysum([1.1, 2.2, 3.3, 4.4]) 11.0
```

中文翻译

有趣的是，我们可以用 Python 的 if/else 三元表达式（在第 12 章介绍过）来节省一些代码空间。我们还可以把它推广到任何“可相加”的类型（如果假设输入至少有一个元素，这会更容易实现），并用扩展解包赋值（extended-unpacking assignment，第 11 章讲过）让“首项/剩余项”的解包更简单。示例 19-2 汇集了这三处修改，随时可以在文件中运行或粘贴到 REPL。示例 19-2. mysumalts.py

```
python
def mysum(L): return 0 if not L else L[0] + mysum(L[1:])
# 使用三元表达式
def mysum(L): return L[0] if len(L) == 1 else L[0] + mysum(L[1:]) # 任意类型，假设至少一项
def mysum(L): first, rest = L return first if not rest else first + mysum(rest) # 使用扩展解包
单独测试时，这三个替代版本处理数字求和的结果与原版完全相同：python
>>> mysum([1, 2, 3, 4, 5]) 15
独特之处在于：后两个版本对空序列会失败（例如 mysum([])），但它们能处理任何支持 + 的对象类型组成的序列，而不仅仅是数字（对于字符串，效果类似于''.join(L)）：python
>>> mysum(('h','a','c',
```

k')) # 后两个版本在 mysum([]) 上会失败'hack'>>> mysum(['hack','app','code']) # 但它们支持非数字类型'hackappcode'你可以自己跑一些测试获得更多洞察。如果你仔细研究这三个变体，还会发现：- 后两个版本可以处理单个字符串参数（例如 mysum('hack')），因为字符串是单字符串组成的序列（虽然这个用例没什么用：你会得到同样的字符串）。- 第三个变体还适用于任意可迭代对象（iterables），包括打开状态下的输入文件（mysum(open(name)))，但其他版本的下标索引通常在非序列对象上会失败（扩展解包的演示见第 14 章）。你可能还会注意到，第三个变体的解包赋值很像函数头中的收集参数（collector），于是忍不住想把它改写成那样。但这并不完全行得通，因为它会期待独立的多个参数，而不是单个可迭代对象——除非我们在顶层输入和递归调用处都加上。下面是最终结果，不过由于它是对离散参数求和，它解决的问题与前面两个版本以及内置 sum 都不一样：python >>> def mysum(first, rest): return first if not rest else first + mysum(rest) >>> mysum([1, 2, 3, 4, 5]) 15 >>> mysum('hack')'hack'最后请记住：递归可以是直接的，就像迄今的例子那样；也可以是间接的，就像下面这样——一个函数调用另一个函数，而后者又回过头来调用前者。净效果相同，只是每一层有两次函数调用而不是一次：python >>> def mysum(L): if not L: return 0 return nonempty(L) # 调用一个会反过来调用我的函数>>> def nonempty(L): return L[0] + mysum(L[1:]) # 间接递归>>> mysum([1.1, 2.2, 3.3, 4.4]) 11.0

深度理解

·核心概念：同一逻辑有不同编码姿态——三元表达式（单行化）、`len(L)==1` 基准（支持任意可加类型）、扩展解包 `first, rest`（支持任意可迭代对象）。三种写法都是“头 + 尾递归”，但泛化程度不同。

·底层实现：`first, rest = L` 在解释器层面会把“除了第一个之外的所有元素”打包成新列表——它要求对象可迭代（有 `iter`），而不要求支持下标（`getitem`）。这就是第三版能对文件对象工作的原因：文件是迭代器，但不可下标访问。而 `L[0]/L[1:]` 需要序列协议（`getitem`），所以前两版限制更多。

·设计原因：作者用这组例子演示“写递归时先想清楚输入的前提假设”——要不要处理空序列？要不要支持非数字？假设不同，基准情形和递推方式就不同。接口的宽容度决定代码的适用面。

·实际问题：`mysum('hack')` 这种把参数列表展开（starred）的写法，在调用任何接受可变参数的函数时都很常见（如 `print(args)`、`pow(pair)`）。而 `rest` 收集参数 + 递归调用再展开，是把“函数式递归”翻译成“参数传递式”的桥梁。

·初学者误区：① 误以为 `first, rest = L` 需要列表——其实它只需要可迭代对象；② 以为 `def mysum(first, rest)` 能直接替代 `def mysum(L)`——函数头收集的是独立参数，不传就无法喂入单个列表，作者特意演示了这个“想当然”会失败的地方；③ 容易忽视间接递归——A 调 B，B 调 A

也是递归，且更难在代码里一眼看出，调试时别漏掉。

·设计思想：三个版本并存而不评判谁最优——Python 的哲学是“有多种做法但明确推荐简单者”。递归用于演示技巧，真正的日常选择仍倾向于 `sum()` 或循环。

19.3.3 循环语句与递归 (Loop Statements Versus Recursion)

英文原文

Though recursion works for summing in the prior sections' examples, it's probably overkill in this context. In fact, recursion is not used nearly as often in Python as in more esoteric languages like Prolog or Lisp, because Python emphasizes simpler procedural statements like loops, which are usually more natural. The while, for example, often makes things more concrete, and it doesn't require that a function be defined to allow recursive calls:

```
>>> L = [1, 2, 3, 4, 5] >>> tot = 0
>>> while L: tot += L[0] L = L[1:] >>> tot
15
```

Better yet, for loops iterate for us automatically, making recursion largely extraneous in many cases (and, in all likelihood, less efficient in terms of memory space and execution time):

```
>>> L = [1, 2, 3, 4, 5] >>> tot = 0
>>> for x in L: tot += x >>> tot
15
```

With looping statements, we don't require a fresh copy of a local scope on the call stack for each iteration, and we avoid the speed costs associated with function calls in general. (

Stay tuned for Chapter 21's timer case study for ways to compare the execution times of alternatives like these.)

中文翻译

尽管递归在前几节的例子中能完成求和，但在这个场景下它很可能是“小题大做”。事实上，递归在 Python 中的使用频率远不及在 Prolog 或 Lisp 这类更晦涩的语言中，因为 Python 强调更简单的过程式语句——比如循环，它通常更自然。例如 while 常常让事情更具体，而且它不需要为了允许递归调用而先定义一个函数：

```
python >>> L = [1, 2, 3, 4, 5] >>> tot = 0 >>> while L: tot += L[0] L = L[1:] >>> tot
```

15 更好的是，for 循环会自动为我们迭代，使得递归在许多情况下基本是多余的（而且很可能在内存空间和执行时间上更低效）：

```
python >>> L = [1, 2, 3, 4, 5] >>> tot = 0 >>> for x in L: tot += x >>> tot
```

15 使用循环语句时，我们不需要为每次迭代在调用栈上准备一份局部作用域的新副本，也避免了函数调用本身的普遍速度开销。（敬请期待第 21 章的计时器案例研究，那里有比较这些替代方案执行时间的办法。）

深度理解

- 核心概念：递归 vs 循环不是“谁对谁错”，而是“何时更合适”。线性（linear）任务——从头到尾扫一遍——用循环更自然；非线性（嵌套/树形）任务才需要递归。
- 底层实现：循环的开销是常数——一个 for 迭代器对象 + 一个循环变量，栈帧只有一份；递归的开销是线性增长——

—每层都新建栈帧、传参、返回。作者明确指出递归在内存空间和执行时间上都不占优。第 21 章会用 timeit 之类的计时器实测验证。

·设计原因：Python 的设计哲学（Prolog/Lisp 的遗产是递归优先，Python 是“过程式优先、函数式为辅”）决定了标准库和惯用法都以循环为主。语言文化塑造编码习惯。

·实际问题：深度不确定的树形结构（目录树、JSON、图）用递归；确定长度的线性处理（求和、逐项转换、统计）用循环。如果递归深度可能超过默认 1000 层，循环往往是更稳的选择。

·初学者误区：初学者常被“递归很酷”吸引而到处滥用——作者的忠告是：递归不是更高级的循环，而是不同形状的工具。线性任务用递归，除了慢，还容易 RecursionError。

19.3.4 处理任意结构 (Handling Arbitrary Structures)

英文原文

On the other hand, recursion—or equivalent and explicit stack-based algorithms we'll explore shortly—can be required to traverse arbitrarily shaped structures. As a simple example of recursion's role in this context, consider the task of computing the sum of all the numbers in a nested sublists structure like this:


```
[1, [2, [3, 4], 5], 6, [7, 8]] # Arbitrarily nested sublists
```

Neither our prior summers nor simple looping statements will work here because this is not a linear iteration. Nested looping statements do not suffice either—because the sublists may be nested to arbitrary depth and in an arbitrary shape, there's no way to know how many nested loops to code to handle all cases.

Instead, the function in Example 19-3 accommodates such general nesting by using recursion to visit sublists along the way.

Example 19-3. `sumtree.py`

```
def sumtree(L, trace=False): tot = 0
    for x in L: # For each item at this level
        if not isinstance(x, list): tot += x
        # Add numbers directly
```

```
    if trace: print(x, end=',')
    else: tot += sumtree(x, trace)
    # Recur for sublists
```

```
    return tot
```

In this file's function, each recursive level runs a for loop to add numbers, or recur into sublists to open new levels. Recall from Chapter 9 that `isinstance` compares object types; it's used here to detect nested sublists.

This code is also instrumented to trace items as they are added to the total: if `trace` is passed a true value, you can see how the object is scanned left to right. Because it also steps down into sublists with recursion along the way, though, the traversal is really both hor

izontal and vertical:

```
>> from sumtree import sumtree >> sumtree([1, [2,
[3, 4], 5], 6, [7, 8]]) 36 >> sumtree([1, [2, [3, 4], 5], 6, [
7, 8]], trace=True) 1, 2, 3, 4, 5, 6, 7, 8, 36
```

中文翻译

另一方面，递归——或者我们稍后就会探索的、与之等价的显式基于栈的算法——对于遍历任意形状的结构可能是必需的。作为递归在此时空角色的简单例子，考虑这个任务：计算一个嵌套子列表（nested sublists）结构中所有数字的和，就像这样：python [1, [2, [3, 4], 5], 6, [7, 8]] # 任意深度的嵌套子列表我们之前的求和函数和简单的循环语句在这里都不管用，因为这不是线性迭代。嵌套循环语句也不够——因为子列表可能嵌套到任意深度、呈现任意形状，你无法知道该写多少层嵌套循环才能覆盖所有情况。取而代之，示例 19-3 中的函数通过使用递归沿途访问子列表，来适配这种普遍存在的嵌套。示例 19-3. sumtree.py python def sumtree(L, trace=False): tot = 0 for x in L: # 遍历这一层的每一项 if not isinstance(x, list): tot += x # 数字直接相加 if trace: print(x, end=',') else: tot += sumtree(x, trace) # 遇到子列表则递归进入 return tot 在这个文件的函数中，每个递归层级运行一个 for 循环来累加数字，或者递归进入子列表来打开新的层级。回顾第 9 章可知，isinstance 用于比较对象类型；在这里它被用来检测嵌套的子列表。这段代码还做了插桩（instrumented）以便追踪累加到总和中的每一项：如果 trace 传入真值，你就能看到对象如何从左到右被扫描。不过，由于它在沿途还会用递归深

入子列表，这次遍历实际上是"横向 + 纵向"双管齐下的：

```
python >>> from sumtree import sumtree >>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]]) 36 >>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]], trace=True) 1, 2, 3, 4, 5, 6, 7, 8, 36
```

深度理解

·核心概念：这是本章最关键的"为什么需要递归"的实证——深度未知的嵌套结构。[1, [2, [3, 4], 5], 6, [7, 8]] 这种树形数据，循环无从下手，递归天然适配：每遇到列表就"再进去一层"。

·底层实现：isinstance(x, list) 做运行时类型检查——它是 C 实现的函数，本质上是检查对象头的类型指针。递归调用 sumtree(x, trace) 每次进入子列表时压入新栈帧，返回时弹栈并累加。注意 trace 参数被逐层传递——它在所有层级共享的是同一份引用（每层都是同一个布尔值对象，只是按值传递的概念）。

·为什么这样设计：这是深度优先遍历（depth-first traversal）——先钻到最深处，再一层层回来。打印顺序 1, 2, 3, 4, 5, 6, 7, 8 表明：[2, [3, 4], 5] 被完全扫完后才轮到 6。结构上"先纵向后横向"，与人类阅读目录树的方式一致。

·实际问题：任何"树"（文件系统、JSON、XML、网站链接图）的遍历都长这样。微信文件夹、思维导图、公司组织架构图——凡是"里面还套着同类型的东西"，就是递归的地盘。

·初学者误区：① 用 type(x) == list 而不是 isinstance——

—前者不认子类，`isinstance` 才是正道；② 认为"嵌套多深都能行"——默认递归上限 1000 层，超深的 JSON 结构会 `RecursionError`；③ 误以为 `trace=True` 是"打印和计算同时进行"——其实是每次累加一个数字后立刻打印，顺序正是遍历顺序。

19.3.5 用独立脚本测试 (Testing with a Separate Script)

英文原文

At this point, we could test other cases by typing the `m` interactively or by adding code to the bottom of the file, but you're probably starting to see that this can be a bit limiting. Instead, the script in Example 19-4 makes the process automatic and easily repeatable. As a bonus, it can be used for other summers we'll code in a moment.

Example 19-4. `sumtreetester.py`

```
tests = ( [1, [2, [3, 4], 5], 6, [7, 8]], # Mixed nesting => 36
          [1, [2, [3, [4, [5]]]]], # Right-heavy nesting => 15
          [1, [2, [3, [4, [5]]]]], # Left-heavy nesting => 15
```

```
def tester(sumtree, trace=True):
    for test in tests:
        print(sumtree(test, trace))
```

To use this tester, simply import both `summer` and `tester`, and pass the former to the latter. When run, our `summer` prints numbers as added with the final sum at the end, for each of three can

ned tests:

```
>> from sumtree import sumtree # Get the summer
>> from sumtreetester import tester # Get the tester
>> tester(sumtree) # Run the tester on the summer 1
, 2, 3, 4, 5, 6, 7, 8, 36 1, 2, 3, 4, 5, 15 1, 2, 3, 4, 5, 15
```

Within tester, sumtree refers to the summer function passed into it. Again, because functions are objects, passing them around this way is natural, and makes code flexible.

中文翻译

此时，我们本来可以交互式输入或用往文件底部添加代码的方式测试其他用例，但你大概已经看出这样有些局限。取而代之，示例 19-4 中的脚本让这个过程的自动化且易于重复。

作为额外收获，它还可以用于我们稍后要写的其他求和函数。

。示例 19-4. sumtreetester.py python tests = ([1, [2, [3, 4], 5], 6, [7, 8]], # 混合嵌套 => 36 [1, [2, [3, [4, [5]]]]], # 右倾嵌套 => 15 [1, [2, [3, [4, [5]]]]], # 左倾嵌套 => 15

def tester(sumtree, trace=True): for test in tests: print(sumtree(test, trace)) 要使用这个测试器，只需同时导入求和函数和测试器，并把前者传给后者。运行时，我们的求和函数会对三组预设用例依次打印累加的数字和最终的和：

```
: python >>> from sumtree import sumtree # 获取求和函数
>>> from sumtreetester import tester # 获取测试器
>>> tester(sumtree) # 对求和函数运行测试器
1, 2, 3, 4, 5, 6, 7, 8, 36 1, 2, 3, 4, 5, 15 1, 2, 3, 4, 5, 15
在 tester 内部，sumtree 指向的是被传入的求和函数。再次强调：因为函数是对象，像这样把它们传来传去非常自然，也让
```

代码更加灵活。

深度理解

·核心概念：函数作为参数传入另一个函数（higher-order function，高阶函数）——`tester(sumtree)` 把函数本身当作数据传递。这是“测试器”能够复用于不同求和实现的原因：同一套用例测试多个实现。

·底层实现：`from sumtree import sumtree` 执行时，导入机制把模块 `sumtree` 命名空间中的名字 `sumtree` 所绑定的函数对象复制到当前命名空间。`tester(sumtree)` 的调用把函数对象的引用压栈并绑定到形参 `sumtree`，之后 `print(sumtree(test, trace))` 就通过这个引用调用它。三个测试用例覆盖了三种嵌套形态（混合、右倾、左倾），验证遍历逻辑的健壮性。

·设计原因：作者用这个小例子提前展示“依赖注入”思想——测试器不知道也不关心求和函数是谁，只要它能接受 `(L, trace)` 并返回总和。这让同一套测试可以无缝切换到接下来的队列版、栈版实现。

·实际问题：真实工程中的测试框架（`pytest`、`unittest`）本质都是这个模式：把待测函数“塞”进一个通用执行器。函数即对象的特性让“传递行为”像“传递数据”一样简单。

·初学者误区：① 传参时写 `tester(sumtree())`（带括号）——这是调用而不是传递，等于传了结果而非函数；② 以为 `tester` 里的 `sumtree` 是全局名字——它是形参，名字只是局部别名。

19.3.6 递归与队列和栈 (Recursion versus Queues and Stacks)

英文原文

It sometimes helps recursion newcomers to understand that internally, Python implements recursion by pushing information on a call stack at each recursive call, so it remembers where it must return and continue later. In fact, it's generally possible to implement recursive-style procedures without recursive calls, by using an explicit stack or queue of your own to keep track of remaining steps.

For instance, Example 19-5 computes the same sums as the prior example, but uses an explicit list to schedule when it will visit items in the subject, instead of using recursive calls. The item at the front of the list is always the next to be processed and summed.

Example 19-5. `sumtreequeue.py`

```
def sumtree(L, trace=False): # Breadth-first, explicit
    queue = list(L) # Start with copy of top level
    tot = 0
    while queue:
        front = queue.pop(0) # Fetch/delete front item
        if not isinstance(front, list):
            tot += front # Add numbers directly
        else:
            if trace: print(front, end=',')
            queue.extend(front)
    # <== Append all in nested list
```

return tot Technically, this code traverses the list in breadth-first fashion (across before down), because it adds nested lists' contents to the end of the list—forming a FIFO (first-in-first-out) queue. The net effect sums by horizontal levels. To test, we can either import and use the new summer directly, or route it to the Example 19-4 tester to be exercised automatically with tracing:

```
>> from sumtreequeue import sumtree # Get the new summer
>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]]) 36
>> from sumtreetester import tester # Unless already imported
>> tester(sumtree) # Run tester on this summer
1, 6, 2, 5, 7, 8, 3, 4, 36 1, 2, 3, 4, 5, 15 5, 4, 3, 2, 1, 15
```

Fine points: we don't have to reimport the tester again if it's already been imported in this session, and importing the same-named summer just works—the new summer's filename makes it unique, and sumtree is always the latest version imported if you import more than one, because imports assign names (see Chapter 18's print emulators for another example of this pattern at work, and watch for more on imports in this book's next part).

More importantly, notice how the order in which numbers are visited here is different than in the original recursive-call version, due to the breadth-first queue. Trace through the tester's tests to see how this pans

out.

If we instead want to emulate the traversal of the recursive-call version more closely, we can change this code to perform depth-first traversal (down before across) simply by adding the contents of nested lists to the front of the list—forming a last-in-first-out (LIFO) stack. Example 19-6 makes the required mods, but the only way it differs from the breadth-first version is the line that adds to the front instead of the end, marked with `<==` in a comment.

Example 19-6. `sumtreestack.py`

```
def sumtree(L, trace=False): # Depth-first, explicit stack
    tot = 0
    items = list(L) # Start with copy of top level
    while items:
        front = items.pop(0) # Fetch/delete front item
        if not isinstance(front, list):
            tot += front # Add numbers directly
```

```
        if trace: print(front, end=',')
        else: items[:0] = front # <== Prepend all in nested list
```

```
    return tot

```

As before, we can use this function directly, or pass it to the same tester; its file makes it distinct, and its name refers to the latest import. When run, this summer visits numbers in the same order as the recursive-calls original, but manages the traversal with an explicit stack instead of recursion:

```
>> from sumtreestack import sumtree # Same name, different file
>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]])
3
```

```
6 >> from sumtreetester import tester # Optional if a  
lready imported >> tester(sumtree) 1, 2, 3, 4, 5, 6, 7,  
8, 36 1, 2, 3, 4, 5, 15 1, 2, 3, 4, 5, 15
```

For more on the last two examples (plus another breadth-first coding variant omitted here), see file `sumtreeetc.py` in the book's examples package. It adds additional tracing so you can watch it walk structures in more detail.

In general, though, once you get the hang of recursive calls, they may be more natural than the explicit scheduling lists they automate, and are generally preferred unless you need to traverse structures in specialized ways. Some programs, for example, perform a best-first search that requires an explicit search queue ordered by relevance or other criteria. If you think of a web crawler that scores sites visited by content, the applications may start to become clearer.

中文翻译

有时候，帮助递归新手理解这一点很有用：在内部，Python 通过在每次递归调用时把信息压入调用栈（call stack）来实现递归，从而记住它必须在何处返回并在稍后继续。事实上，通常完全可以在没有递归调用的情况下实现递归风格的流程——只要用一个你自己的显式栈（stack）或队列（queue）来跟踪剩余步骤。例如，示例 19-5 计算与前一示例相同的和，但它使用一个显式列表来安排何时访问对象中的各项，而不是发出递归调用。列表最前面的项总是下一个要处理并累加的项。示例 19-5. `sumtreequeue.py` python

n def sumtree(L, trace=False): # 广度优先，显式队列tot = 0 items = list(L) # 从顶层副本开始while items: front = items.pop(0) # 取出并删除最前面的项if not isinstance(front, list): tot += front # 数字直接相加if trace: print(front, end=',') else: items.extend(front) # <== 把嵌套列表中的全部内容追加到末尾return tot 严格来说，这段代码以广度优先（breadth-first，先横向后纵向）的方式遍历列表，因为它把嵌套列表的内容添加到列表的末尾——形成一个 FIFO（先进先出）队列。净效果是按水平层级求和。要测试，我们可以直接导入并使用这个新求和函数，也可以把它交给示例 19-4 的测试器自动带追踪运行：python >>> from sumtreequeue import sumtree # 获取新求和函数>>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]]) 36 >>> from sumtreetester import tester # 如果已导入则不必重复>>> tester(sumtree) # 对这个求和函数运行测试器1, 6, 2, 5, 7, 8, 3, 4, 36 1, 2, 3, 4, 5, 15 5, 4, 3, 2, 1, 15

细节说明：如果测试器在本会话中已经导入，我们就不必重新导入；而导入同名的求和函数也能正常工作——新求和函数的文件名使它独一无二，如果你导入了多个版本，sumtree 始终指向最近导入的那一个，因为导入就是给名字赋值（第 18 章的 print 模拟器是这个模式生效的另一个例子，本书下一部分还会有更多关于导入的内容）。更重要的是，注意这里数字被访问的顺序与原始递归调用版本不同——原因是广度优先队列。对照测试器里的用例跟踪一下，看看结果如何。如果我们想要更贴近递归调用版本的遍历方式，可以修改这段代码以执行深度优先（depth-first，先纵向后横向）遍历——只需把嵌套列表的内容添加到列表的最前面——形成一个 LIFO（后进先出）栈。示例 19-6 做

了必要的修改，但它与广度优先版本唯一的区别就是那行"加到前面而非末尾"的代码，用 `<=` 注释标出。示例 19-6.

```
sumtreestack.py python def sumtree(L, trace=False):
# 深度优先，显式栈
tot = 0
items = list(L) # 从顶层副本开始
while items:
    front = items.pop(0) # 取出并删除最前面的项
    if not isinstance(front, list):
        tot += front # 数字直接相加
    if trace:
        print(front, end=',')
    else:
        items[:0] = front # <= 把嵌套列表中的全部内容前插到最前面
return tot
```

和之前一样，我们可以直接使用这个函数，或把它传给同一个测试器；它的文件名让它与众不同，而它的名字指向最新导入的版本。运行时，这个求和函数以与原始递归调用版本相同的顺序访问数字，但用显式栈而不是递归来管理遍历：

```
python >>> from sumtreestack import sumtree
# 同名，不同文件
>>> sumtree([1, [2, [3, 4], 5], 6, [7, 8]])
36
>>> from sumtreetester import tester # 已导入则为可选项
>>> tester(sumtree)
1, 2, 3, 4, 5, 6, 7, 8, 36
1, 2, 3, 4, 5, 15
1, 2, 3, 4, 5, 15
```

关于最后两个示例的更多内容（以及此处省略的另一个广度优先编码变体），参见本书示例包中的 `sumtreeetc.py` 文件。它增加了额外的追踪，让你能更详细地观察它如何行走结构。不过总体而言，一旦你掌握了递归调用，它可能比它所自动化的显式调度列表更自然，因此除非你需要以特殊方式遍历结构，否则通常更受青睐。例如，有些程序执行最佳优先（best-first）搜索，这需要一个按相关性或其他标准排序的显式搜索队列。如果你想到一个按内容给访问过的网站打分的 Web 爬虫，这些应用就会变得更清晰。

深度理解

·核心概念：这是数据结构课的浓缩版——递归只是隐式地使用了调用栈；显式队列/栈可以复现同样遍历。两者唯一的差别是：嵌套列表的内容被放到待处理列表的末尾（FIFO 队列 → 广度优先）还是开头（LIFO 栈 → 深度优先）。

·底层实现：`items.pop(0)` 弹出列表头元素——注意这是 $O(n)$ 操作（列表是数组，弹出头部后所有元素要前移）；`items.extend(front)` 追加到末尾 $O(k)$ ；`items[:0] = front` 是“前插”技巧——把切片 0 位置替换为 `front` 的全部元素。对比递归版：递归的“栈”就是 C 层的调用栈，无需手动管理，但深度受限。

·为什么这样设计：作者想点破一个真相：递归并不神秘。递归版 `sumtree` 之所以输出 1,2,3,4,5,6,7,8，是因为它先钻入 [2, [3, 4], 5] 完成整个子树；栈版用“后进先出”模拟了同样的“先深入再返回”；队列版则先处理完所有兄弟再深入，所以输出变成 1, 6, 2, 5, 7, 8, 3, 4。

·实际问题：广度优先是“按层推进”——社交网络的好友推荐（一度人脉、二度人脉）、最短路径搜索（BFS）；深度优先是“一条路走到黑”——迷宫求解、语法分析。需要自定义排序的“最佳优先”（如爬虫按内容相关度排序待抓 URL）只能用显式队列。

·初学者误区：① 以为递归和显式栈“等价到输出都相同”——只有栈版才等价，队列版是另一种遍历顺序；② 用 `pop(0)` 模拟队列在 Python 里很低效，工程上应使用 `coll`

actions.dequeue ($O(1)$ 两头操作) ; ③ 忘了 `items = list(L)` 复制——直接 `items = L` 会共享原列表, 后续 `pop/extend` 会破坏调用者数据。

·设计思想: 作者用 `<=` 一行之差演示"数据结构的选择决定算法行为"——这是编程里最优雅的杠杆之一: 一个字符的决定 (`front/end`) , 换来完全不同的遍历顺序。

19.3.7 环、路径与栈限制 (Cycles, Paths, and Stack Limits)

英文原文

As is, these programs suffice as demos, but larger recursive applications can sometimes require a bit more infrastructure than shown here: they may need to avoid cycles or repeats, record paths taken for later use, and expand stack space when using recursive calls instead of explicit queues or stacks. For instance, neither the recursive-call nor the explicit queue/stack examples in this section do anything about avoiding cycles—visiting a location already visited. That's not required here, because we're traversing strictly hierarchical trees of list objects. If data can be a cyclic graph, though, both these schemes will fail: the recursive-call scheme will fall into an infinite recursive loop (and may run out of call-stack space), and the others will fall into simple infinite loops, re-adding the same items to their lists (and may or may not run out of general

memory). In fact, it's easy to demo the perils by creating a cyclic object with the strange code we met in an exercise at the end of Chapter 3:

```
>>> L = [1, 2] >>> L.append(L) # Make a cyclic object: L references itself >>> L [1, 2, [...]]
```

```
>>> from sumtree import sumtree >>> sumtree(L) RecursionError: maximum recursion depth exceeded
```

```
>>> from sumtreequeue import sumtree >>> sumtree(L) ...hang or crash, and ditto for stack...
```

Some programs also need to avoid repeated processing for a state reached more than once, even if that wouldn't lead to a loop. To do better, a recursive-call traversal might make and pass along a mutable set, dictionary, or list of states visited so far and check for repeats as it goes. We will use this scheme in later recursive examples in this book:

```
if state not in visited: visited.add(state) # x.add(state)
, x[state]=True, or x.append(state)
```

...proceed... Nonrecursive alternatives might similarly avoid adding states already visited with code like the following. Subtly, object cycles may require `is (not in)`, and simply checking for duplicates already on the items list would avoid scheduling a state twice but would not prevent revisiting a state visited earlier and hence removed from that list:

```
visited.add(front)
```

```
...proceed... items.extend([x for x in front if x not in visited])
```

This model doesn't quite apply to this section's use case that simply adds numbers in lists, but other applications will generally be able to identify repeated states—a URL of a previously visited web page, for instance. In fact, we'll use such techniques to avoid cycles and repeats in the later examples listed in the next section. Some programs may also need to record complete paths for each state followed so they can report solutions when finished. In such cases, each item in the nonrecursive scheme's stack or queue may be a full path list that suffices for a record of states visited, and contains the next item to explore at either end. Also note that standard Python limits the depth of its runtime call stack—crucial to recursive-call programs—to trap infinite recursion errors. To expand it for deeper journeys, use the `sys` module:

```
>>> sys.getrecursionlimit() # 1000 calls deep default 1000
```

```
>>> sys.setrecursionlimit(10000) # Allow deeper nesting
>>> help(sys.setrecursionlimit) # Read more about it
The maximum allowed setting can vary per platform. This isn't required for programs that use stacks or queues to avoid recursive calls and gain more control over the traversal process (though they also won't catch infinite loops).
```


中文翻译

就目前而言，这些程序足以作为演示，但更大的递归应用有时需要的"基础设施"比这里展示的更多：它们可能需要避免环（cycles）或重复（repeats），记录走过的路径以备后用，并且在使用递归调用而不是显式队列或栈时扩大栈空间。例如，本节中的递归调用版和显式队列/栈版都没有做任何避免环（即访问已访问过的位置）的处理。这里不需要，因为我们遍历的是严格层级化的列表对象树。但如果数据可能是循环图（cyclic graph），这两种方案都会失败：递归调用方案会陷入无限递归循环（并可能耗尽调用栈空间），其余方案则会陷入简单的无限循环，把相同的项不断重新加入它们的列表（可能耗尽、也可能不耗尽一般内存）。事实上，用我们在第 3 章末尾练习中见过的古怪代码创建一个循环对象，很容易就能演示这种危险：

```
python >>> L = [1, 2] >>> L.append(L) # 制造循环对象：L 引用自身 >>> L [1, 2, [...]] >>> from sumtree import sumtree >>> sumtree(L) RecursionError: maximum recursion depth exceeded >>> from sumtreequeue import sumtree >>> sumtree(L) ...挂起或崩溃，栈版本同样如此... 有些程序还需要避免对一个"到达过一次以上"的状态重复处理，即使这不会导致死循环。要做到更好，递归调用遍历可以创建并传递一个可变的集合（set）、字典或列表，记录迄今访问过的状态，边走边检查是否重复。本书后面的递归示例会用到这个方案：
```

```
python if state not in visited: visited.add(state) # x.add(state)、x[state]=True 或 x.append(state) ...继续处理... 非递归的替代方案也可以类似地避免加入已访问的状态，代码如下。微妙之处在于：对象环可能要求使
```

用 `is` (而不是 `in`) ; 而且仅检查 `items` 列表中已有的重复项, 虽然能避免一个状态被调度两次, 却不能防止重新访问一个更早访问过、因而已被移出该列表的状态: `python visited.add(front) ...继续处理... items.extend([x for x in front if x not in visited])` 这个模型并不完全适用于本节这个"简单地把列表中的数字相加"的用例, 但其他应用通常都能识别重复状态——例如, 一个以前访问过的网页的 URL。

事实上, 我们将在下一节列出的后面示例中使用这些技术来避免环和重复。有些程序可能还需要记录每个被跟踪状态的完整路径, 以便结束时报告解决方案。在这种情况下, 非递归方案中栈或队列里的每个项, 都可以是一份完整的路径列表——它既足以作为已访问状态的记录, 又在其两端包含下一个要探索的项。还要注意, 标准 Python 会限制其运行时调用栈的深度——这对递归调用程序至关重要——以捕获无限递归错误。要扩大它以支持更深的旅途, 请使用 `sys` 模块: `python >>> sys.getrecursionlimit() # 默认 1000` 层调用深度1000 `>>> sys.setrecursionlimit(10000) # 允许更深的嵌套` `>>> help(sys.setrecursionlimit) # 了解更多` 允许设置的最大值因平台而异。对于使用栈或队列来避免递归调用、从而对遍历过程获得更多控制权的程序, 这不是必需的 (不过它们同样也拦截不了无限循环) 。

深度理解

·核心概念: 递归遍历的三大"事故现场": 环 (对象引用自身 → 无限循环)、重复状态 (同一状态处理多次 → 浪费)、栈溢出 (深度超限 → `RecursionError`)。对策分别是: `visited` 集合、`is/in` 去重、调大递归上限或用显式栈

。

·底层实现：L.append(L) 制造了一个自引用对象——L 里存着指向自身的引用，所以打印显示 [1, 2, [...]] (... 表示"还是我")。对递归版，每一层都再进一层、永不到底，直到撞上 RecursionError (CPython 默认上限 1000)；对队列/栈版，则不断把相同的项重新加入列表，永不终止——比报错更糟，因为程序会静默挂死。RecursionError 本质上是 C 层检查 PyEnterRecursiveCall 深度计数后的主动止损。

·为什么这样设计：Python 设置递归上限是为了把"无限递归导致内存耗尽"变成"可捕获的异常"。sys.setrecursionlimit 可以调高，但真正想遍历极深结构，工程做法是转写为显式栈/队列——完全绕开调用栈限制，还省去每层的函数调用开销。

·实际问题：Web 爬虫必须记录已访问 URL（否则在链接图中死循环）；图算法（如 DFS 找连通分量）必须用 visited 集合；回溯算法（八皇后、数独）必须记录路径。这些"防重、记路、控深"三件套是所有复杂遍历的标配。

·初学者误区：① 以为 RecursionError 是"程序 bug 的名字"——它也可能是"数据太深"，两者要学会分辨；② 用 in（相等性比较）去重引用环——两个相等的对象可能不是同一个对象，循环引用场景下要用 is（身份比较）或 id 集合；③ 以为 setrecursionlimit 可以无限调大——每层栈帧占真实内存，调太大照样崩（可能直接段错误，比异常更可怕）。

19.3.8 更多递归示例 (More Recursion Examples)

英文原文

Although this section's example is artificial, it is representative of a larger class of programs; inheritance trees and module import chains, for example, can exhibit similarly general structures, and computing tools such as permutations can require arbitrarily many nested loops. In fact, we'll use recursion again in such roles later in this book: - In Chapter 20's `permute.py`, to shuffle arbitrary sequences - In Chapter 25's `reloadall.py`, to traverse import chains - In Chapter 29's `classtree.py`, to traverse class inheritance trees - In Chapter 31's `lister.py`, to traverse class inheritance trees again - In Appendix B, "Solutions to End-of-Part Exercises" at the end of this part of the book: `countdowns` and `factorials` The second and third of these will also detect states already visited to avoid cycles and repeats. Although simple loops should generally be preferred to recursion for linear iterations on the grounds of simplicity and efficiency, you'll find that recursion is essential in scenarios like those in these later examples. Moreover, you sometimes need to be aware of the potential of unintended recursion in your programs. As you'll also see later in the book, some operator-overloading methods in classes such as `setattr` and `getattr`

tribute and even repr have the potential to recursively loop if used incorrectly. Recursion is a powerful tool, but it tends to be best when both understood and expected!

中文翻译

虽然本节示例是人为构造的，但它代表了一类更大的程序；例如，继承树 (inheritance trees) 和模块导入链 (import chains) 会表现出同样通用的结构，而排列 (permutations) 等计算工具可能需要任意多层嵌套循环。事实上，本书后面还会在类似的角色中再次使用递归：- 在第 20 章的 `permute.py` 中，用来打乱任意序列- 在第 25 章的 `reload all.py` 中，用来遍历导入链- 在第 29 章的 `classtree.py` 中，用来遍历类继承树- 在第 31 章的 `lister.py` 中，再次用来遍历类继承树- 在本部分末尾的附录 B 《分部练习解答》中：倒数与阶乘其中第二和第三个还会检测已访问状态以避免环和重复。尽管对于线性迭代，出于简洁和效率的考虑，通常应该优先选择简单循环而非递归，但你会发现在这些后面的示例场景中，递归是必不可少的。此外，有时你还需要警惕程序中无意中的递归。正如你后面会看到的，类中的一些运算符重载方法——例如 `setattr` 和 `getattr`，甚至 `repr`——如果使用不当，有可能递归循环。递归是一个强大的工具，但它在“被理解、且被预期”时表现最佳！

深度理解

·核心概念：递归的典型应用领域清单——排列生成（组合数学）、导入链遍历（模块依赖图）、类继承树遍历（MRO 可视化）。这些结构的共同点：深度未知 + 形状不可

预测。

·底层实现：repr 的递归陷阱值得展开——如果你在 repr 里打印对象自身（比如 `return f'{self.x} {self}'`），打印时解释器调用 `repr` → 它又格式化 `self` → 又调用 `repr`.....无限循环直到 `RecursionError`。setattr 同理：在 setattr 里写 `self.x = 1` 会再次触发 setattr，正确写法是用 `object.setattr(self, 'x', 1)`。

·为什么这样设计：递归的价值不在“能省代码”，而在“结构自动适配”——树的层数变深时，递归代码一个字都不用改。排列问题需要“任意多层嵌套循环”，手写循环层数是写不出来的，递归天然满足。

·实际问题：调试时最怕“意料之外的递归”——程序莫名 `RecursionError` 或卡死，往往不是数据太深，而是某个魔术方法（dunder method）在隐式调用自己。遇到诡异的栈溢出，先查 `repr/eq/hash` 是否间接递归。

·初学者误区：① 以为递归“只能用来炫技”——真实项目里遍历目录（`os.walk` 内部就是迭代，但手写目录树遍历常用递归）、解析 JSON 都靠它；② 忽视间接递归——A 调 B、B 调 C、C 调 A，三处代码分开看毫无问题，合起来就是死循环；③ 把 repr 当成 print 的替代品乱用，结果自食其果。

19.4 函数工具：属性、注解等 (Function Tools: Attributes, Annotations, Etc.)

英文原文

Let's move on to a category of tools that may seem less ethereal than recursion to some earthlings. As we've seen in this part of the book, functions in Python are much more than code-generation specifications for a compiler—they are full-blown objects, stored in pieces of memory all their own. As such, they can be freely passed around a program and called indirectly. They also support operations that have little to do with calls at all, including attributes and annotation. We've sampled some of these topics earlier, but this section provides expanded coverage.

中文翻译

让我们转向一类对某些“地球人”来说可能不像递归那么玄乎的工具。正如我们在本书本部分所看到的，Python 中的函数远不只是给编译器的代码生成规范——它们是完整意义上的对象（objects），存储在自己专属的内存块中。因此，它们可以在程序中自由传递并被间接调用。它们还支持一些与调用几乎无关的操作，包括属性（attributes）和注解（annotations）。我们之前已经浅尝过其中一些主题，本节将提供更全面的讲解。

深度理解

·核心概念：一句话——函数是对象。def 不只创建“代码”，它创建的是“一个内存中的对象 + 一个名字绑定”。任何对象能做的事（传递、存储、检查、挂属性），函数都能

做。

·底层实现：在 CPython 中，每个函数对象是 PyFunctionObject 结构体实例，内含 funccode（字节码）、funcname、funcdefaults、funcannotations、dict 等字段。def 执行时只是：创建函数对象 → 把名字绑定到它（本质是一次 = 赋值）。

·为什么这样设计：函数即对象是函数式编程的地基——map/filter/reduce 把函数当参数传，装饰器把函数当返回值，回调把函数存进结构。没有"一等函数"，这些全都无从谈起。

·实际问题：调试、测试、框架（Flask 路由、pytest fixtures、Django 视图）都大量依赖"把函数当成可配置的数据"。理解了函数即对象，你才能读懂这些框架的魔法。

·初学者误区：初学者常把"函数"想象成"一段代码"，而忘了它同时是"一个可传递的实体"。看到一个函数被赋值给变量、塞进列表时感到困惑，根源就是没建立"函数即对象"的心智模型。

19.4.1 一等对象模型 (The First-Class Object Model)

英文原文

Because Python functions are objects, you can write programs that process them generically. Function objects may be reassigned to other names, passed to other functions, embedded in data structures, returned

from one function to another, and more, as if they were simple numbers or strings. Function objects happen to support a special operation—they can be called by listing arguments in parentheses—but they belong to the same general category as other objects. As we've seen, this is usually called a first-class object model; it's ubiquitous in Python, and a necessary part of functional programming. We'll explore this programming mode more fully in this and the next chapter; because its motif is founded on the notion of applying functions, it treats functions as a kind of data. We've explored some generic use cases for functions in earlier examples, but a quick review helps to underscore the model. For example, there's really nothing special about the name used in a `def` statement: it's just a variable assigned in the current scope, as if it had appeared on the left of an `=` sign. Because the function name is simply a reference to an object after a `def` runs, you can reassign that object to other names freely and call it through any reference:

```
>>> def exclaim(message): # Name exclaim assigned to function object
print(message + '!') >>> exclaim('Direct call') # Call object through original name
Direct call!
```

```
>>> x = exclaim # Now x references the function too
>>> x('Indirect call') # Call object through name x by adding ()
Indirect call!
```

And because arguments are passed by assigning objects, it's just as easy to pass functions to other functions as arguments. The callee may then call the passed-in function just by adding arguments in parentheses (see the earlier summer tester in Example 19-4 for another example of this pattern):

```
>>> def generic(func, arg): func(arg) # Call the passed-in object by adding ()
```

```
>>> generic(exclaim,'Argument call') # Pass the function to another function Argument call!
```

You can even stuff function objects into data structures, as though they were integers or strings. The following, for example, embeds the function twice in a list of tuples, as a sort of actions table. Because Python compound types like these can contain any sort of object, there's no special case here, either (Example 18-2 used similar code):

```
>>> schedule = [ (exclaim,'Hack'), (exclaim,'Code') ]
>>> for (func, arg) in schedule: func(arg) # Call functions embedded in containers Hack! Code!
```

This code simply steps through the schedule list, calling the exclaim function with one argument each time through. As we saw in Chapter 17's examples, functions can also be created and returned for use elsewhere—the closure created in this mode also retains state from the enclosing scope:

```
>>> def implore(verb): # Make a function but don't
call it def exclaim(noun): print(f'{verb} {noun}!') return
exclaim >>> I = implore('Hack') # Label in enclosing
scope is retained >>> I('Code') # Call the function th
at make returned Hack Code!
```

```
>>> I('App') Hack App!
```

Python's first-class object model and lack of type constraints make for a very flexible programming language.

中文翻译

因为 Python 函数是对象，你可以编写程序通用地处理它们。函数对象可以被重新赋值给其他名字、传给其他函数、嵌入数据结构、从一个函数返回到另一个函数，等等——就好像它们是简单的数字或字符串一样。函数对象恰好还支持一个特殊操作——把参数列在括号里就可以调用它们——但它们与其他对象属于同一个大类。正如我们所见，这通常被称为一等对象模型（first-class object model）；它在 Python 中无处不在，也是函数式编程的必要组成部分。我们将在本章和下一章更充分地探索这种编程模式；因为它的主旋律建立于“应用函数”这一概念之上，它把函数当作一种数据来对待。我们已经在更早的例子中探索过函数的一些通用用例，但快速复习一下有助于加深对这个模型的理解。例如，def 语句中使用的名字其实没什么特别之处：它只是当前作用域中赋值的一个变量，就好像它出现在 = 号的左侧一样。由于 def 运行后函数名只是对一个对象的引用，你可以自由地把那个对象重新赋值给其他名字，并通过任何引用调用它：python >>> def exclaim(message): # 名字

exclaim 绑定到函数对象 `print(message + '!')` `>>> exclaim('Direct call')` # 通过原名调用对象 `Direct call!` `>>> x = exclaim` # 现在 `x` 也引用这个函数 `>>> x('Indirect call')` # 通过名字 `x` 加 `()` 调用对象 `Indirect call!` 而且因为参数是通过“赋值对象”的方式传递的，把函数作为参数传给其他函数同样轻而易举。被调用者只需在括号里加上参数就能调用传入的函数（示例 19-4 里那个求和函数测试器就是这种模式的又一个例子）：`python >>> def generic(func, arg): func(arg)` # 加上 `()` 调用传入的对象 `>>> generic(exclaim, 'Argument call')` # 把函数传给另一个函数 `Argument call!` 你甚至可以把函数对象塞进数据结构，就好像它们是整数或字符串一样。例如，下面这段代码把函数嵌入了两次，放进一个元组列表里，作为一种动作表（actions table）。因为 Python 的复合类型可以容纳任何种类的对象，这里也没有任何特殊情况（示例 18-2 用过类似的代码）：`python >>> schedule = [ (exclaim,'Hack'), (exclaim,'Code') ]` `>>> for (func, arg) in schedule: func(arg)` # 调用嵌入在容器里的函数 `Hack! Code!` 这段代码只是遍历 `schedule` 列表，每次用同一个参数调用 `exclaim` 函数。正如我们在第 17 章的示例中看到的，函数也可以被创建并返回供他处使用——以这种方式创建的闭包（closure）会保留来自包围它的作用域的状态：`python >>> def implore(verb): # 制造一个函数，但不调用它` `def exclaim(noun): print(f'{verb} {noun}!')` `return exclaim` `>>> I = implore('Hack')` # 包围作用域中的标签被保留 `>>> I('Code')` # 调用 `make` 返回的函数 `Hack Code!` `>>> I('App')` `Hack App!` Python 的一等对象模型和缺乏类型约束的特点，造就了一种非常灵活的编程语言。

深度理解

·核心概念：一等函数 = 函数能和整数、字符串一样被赋值、传参、存容器、作返回值。本节用四个递进例子演示：改名调用 → 传参调用 → 容器存储 → 闭包返回。这是通往所有函数式技巧的大门。

·底层实现：def 只是名字绑定。generic(exclaim, 'Argument call') 执行时，解释器把 exclaim 这个函数对象的引用压栈并绑定到 generic 的形参 func，func(arg) 则通过该引用发起调用。implore('Hack') 返回的 exclaim 内部持有 verb='Hack' 的闭包单元格 (cell) ——Python 用 LOADDEREF 字节码读取它，这就是"状态被保留"的底层机制。

·为什么这样设计：GUI 事件回调、排序关键字 (sorted(key=func))、装饰器、依赖注入.....全都依赖"把行为当作数据传递"。一等函数让代码可以参数化行为，而不是复制行为。

·实际问题：schedule 列表就是"命令模式"的雏形——把"动作 + 参数"打包成数据，统一循环执行。事件系统、菜单系统、状态机都是这个模式。

·初学者误区：① 传函数忘了加括号 vs 加了括号——传 func 是传引用，传 func() 是传结果，一字之差天壤之别；② 以为 x = exclaim 是"复制了函数"——不，只是复制了引用，函数对象只有一个；③ 闭包里 verb 看起来像全局变量，其实它既不是局部也不是全局，而是"自由变量" (free variable) ——这在第 17 章是重点，此处是复习

19.4.2 函数内省 (Function Introspection)

英文原文

In fact, functions are more flexible than you might expect. Because they are objects, we can also process functions with normal object tools. For instance, by now we know that once we make a function we can call it as usual:

```
>> def func(a):  
b = 'Hack' return b a >> func(8)'HackHackHackHackHackHackHackHack'
```

But the call expression is just one of a set of operations defined to work on function objects. For instance, we can also inspect their attributes generically:

```
>> func.name'func'>> dir(func)'annotations','builtins','call','class','closure',  
...more omitted: 38 total...'setattr','sizeof','str','subclasshook','typeparams']
```

Introspection—internals access—tools like this allow us to explore implementation details. For example, functions have attached code objects, which provide details on aspects such as the functions' local variables and arguments:

```
>> func.code <code object func at 0x103ef3910, file
"<stdin>", line 1> >> dir(func.code) ['class','delattr','
dir','doc','eq','format','ge',
...more omitted: 48 total...'coposonlyargcount','coqua
lname','costacksize','covarnames','replace'] >> func.c
ode.covarnames ('a','b') >> func.code.coargcount 1
```

Tool writers can make use of such information to manage functions—in fact, we will too in Chapter 39, to implement validation of function arguments with the decorators introduced ahead. Whether you code or use such tools, object introspection boosts function utility.

中文翻译

事实上，函数比你想象的还要灵活。因为它们是对象，我们也可以用普通的对象工具来处理函数。例如，到目前为止我们已经知道，一旦创建了一个函数，就可以像平常一样调用它：python >>> def func(a): b='Hack' return b a >>> func(8)'HackHackHackHackHackHackHackHack'但调用表达式只是针对函数对象定义的一整套操作中的一项。例如，我们还可以通用地检查它们的属性：python >>> func.name'func'>>> dir(func)'annotations','builtins','call','class','closure', ...省略更多：共 38 个...'setattr','sizeof','str','subclasshook','typeparams'] 内省 (introspection)——即访问内部信息——这类工具让我们能够探索实现细节。例如，函数挂载着代码对象 (code objects)，它们提供了函数局部变量、参数等细节信息：python >>> func.code <code object func at 0x103ef3910, file "<stdin

```
>", line 1> >>> dir(func.code) ['class','delattr','dir','doc','eq','format','ge', ...省略更多: 共 48 个...'coposonly','argcount','coqualname','costacksize','covarnames','replace'] >>> func.code.covarnames ('a','b') >>> func.code.coargcount
```

1 工具作者可以利用这类信息来管理函数——事实上，我们也会在第 39 章这样做：用前面介绍的装饰器（decorators）实现函数参数的校验。无论你编写还是使用这类工具，对象内省都提升了函数的实用价值。

深度理解

·核心概念：内省（introspection）= 程序在运行时"检查自己"的能力。函数暴露了 name、code、annotations、defaults 等属性，让元编程（metaprogramming）成为可能。

·底层实现：func.code 指向 PyCodeObject（代码对象），其中 covarnames 是局部变量名元组（这里 ('a','b') 包含参数 a 和局部变量 b）、coargcount 是位置参数个数（1）、cofilename/cofirstlineno 记录来源文件与行号、coconsts 是常量表。这些是编译器在字节码生成阶段就填好的静态信息。

·为什么这样设计：把"代码自身的元数据"作为普通属性暴露，使 Python 可以实现运行时反射（reflection）——框架据此自动生成文档、校验参数、注入依赖，而无需额外维护"描述文件"。Python 的哲学是"数据与元数据都在对象上，随时可取"。

·实际问题：调试器（pdb）、性能分析器（cProfile）、

文档生成器 (pydoc)、参数校验框架都在消费这些属性。inspect 标准库就是对 code 等底层的友好封装。写装饰器时读取 func.name、func.defaults 是日常操作。

·初学者误区：① 以为 dir() 是“调试魔法”——它只是列出对象的全部属性名，是内省的最基础工具；② 混淆 name（函数名）与 code.covarnames（局部变量名）；③ 以为这些信息只能读——事实上 func.defaults 等属性可写，高级框架会改写它们来篡改函数行为（慎用）。

19.4.3 函数属性 (Function Attributes)

英文原文

Nor are function objects limited to the system-defined attributes of the prior section: it's also possible to attach arbitrary user-defined attributes to them. This topic was introduced in Chapter 17, but this section expands on it with additional context and examples. As usual in Python, function attributes are created by simple assignments:

```
>> def func(): pass >> func
<function func at 0x1043771a0> >> func.count = 0
>> func.count += 1 >> func.count 1 >> func.handles
='Button-Press'>> func.handles'Button-Press'>> dir(func)
...most double-underscore names omitted...'str','subclasshook','typeparams','count','handles']
```

Python's own implementation-related data stored on functions follows naming conventions that prevent them from clashing with the more arbitrary attribute names you might assign yourself. Specifically, all function internals' names have leading and trailing double underscores ("X"):

```
>> len(dir(func)) 40 >> [x for x in dir(func) if not x.startswith('_')] ['count', 'handles']
```

If you're careful not to name attributes the same way as Python, you can safely use the function's namespace as though it were your own namespace or scope. Naturally, all of this works the same for functions made with lambda:

```
>> F = lambda: None >> len(dir(F)) 38 >> F.book = 'LP6E' >> F.book 'LP6E'
```

As covered in Chapter 17, such attributes can be used to attach state information to function objects directly, instead of using other techniques such as globals, nonlocals, and classes. Unlike nonlocals, such attributes are accessible anywhere the function itself is, even from outside its code.

In a sense, this is also a way to emulate "static locals" in other languages—variables whose names are local to a function, but whose values are retained after a function exits. Attributes are related to objects instead of scopes (and must be referenced through the function name within its code), but the net effect is similar.

ar.

Moreover, as also explored in Chapter 17, when attributes are attached to functions generated by other factory functions, they also support multiple copy, writable, and per-call state retention, as an alternative to closures and class-instance attributes. This makes function attributes a broadly useful tool—like the topics of the next section.

中文翻译

函数对象也并非局限于上一节的系统定义属性：你还可以给它们附加任意用户自定义的属性。这个话题在第 17 章引入过，但本节将用更多的背景和示例展开它。和 Python 中通常的做法一样，函数属性通过简单的赋值语句创建：

```
python >>> def func(): pass >>> func <function func at 0x1043771a0> >>> func.count = 0 >>> func.count += 1 >>> func.count 1 >>> func.handles = 'Button-Press' >>> func.handles 'Button-Press' >>> dir(func) ...大多数双下划线名字已省略... 'str', 'subclasshook', 'typeparams', 'count', 'handles']
```

Python 存储在函数上的、与实现相关的数据遵循一套命名约定，以防止它们与你自行赋值的、更随意的属性名冲突。具体来说，所有函数内部的名字都带前导和尾随的双下划线（"X"）：

```
python >>> len(dir(func)) 40 >>> [x for x in dir(func) if not x.startswith('_')] ['count', 'handles']
```

只要你小心不把自己的属性命名为 Python 保留的样式，就可以安全地把函数的命名空间当作你自己的命名空间或作用域来用。自然，这一切对用 lambda 创建的函数同样有效：

```
python >>> F = lambda: None >>>
```

`len(dir(F))` 38 >>> `F.book = 'LP6E'` >>> `F.book` 'LP6E'正如第 17 章所述，这类属性可以用来把状态信息直接附加到函数对象上，而不必使用全局变量、`nonlocal` 或类等其他技术。与 `nonlocal` 不同，这类属性在函数本身所能到达的任何地方都可以访问——即使是在函数代码之外。在某种意义上，这也是模拟其他语言中“静态局部变量”（`static locals`）的一种方式——这些变量的名字局限于函数内部，但其值在函数退出后仍然保留。属性与对象相关而不是与作用域相关（在函数代码内必须通过函数名来引用它们），但净效果是类似的。此外，正如第 17 章也探索过的，当属性被附加到由其他工厂函数（`factory functions`）生成的函数上时，它们还支持多份拷贝、可写、以及每次调用各自独立的状态保留——作为闭包和类实例属性的替代方案。这使函数属性成为一个用途广泛的工具——就像下一节的主题一样。

深度理解

- 核心概念：函数有一个公开的“杂物抽屉”——`func.anything = value` 想挂什么挂什么。这是“给行为附加元数据/状态”最轻量化的方式。
- 底层实现：每个函数对象有一个 `dict` 字典（CPython 的 `funcdict` 字段）。`func.count = 0` 本质就是 `func.dict['count'] = 0`。系统属性（`name`、`code` 等）则存储在结构体的专用字段里，不走 `dict`——这就是 `dir(func)` 里既有 `name` 又有 `count` 的原因。命名约定 X 双重下划线避免了用户属性与系统字段的冲突。
- 为什么这样设计：作者点出的关键对比：`nonlocal` 变量

只能从函数内部访问，函数属性从任何有函数引用的地方都能访问。所以函数属性适合"把状态和函数一起打包传递"——就像给包裹贴上标签，走到哪都能看。

·实际问题：函数属性在真实框架中处处可见——Flask 用 `func.name` 做路由名、`pytest` 用函数属性存标记（marks）、Tkinter 控件回调常挂数据。它也常被用来做"函数级缓存标志"或"调用计数"。

·初学者误区：① 以为属性必须预先"声明"——Python 不需要，任何赋值即创建；② 在函数内部写 `count = 0` 只会创建局部变量，必须写 `func.count`（或 `type(func).count`）才能挂到函数对象上；③ 忘了这是共享单例状态——同一个函数对象被多线程并发调用时，`func.count += 1` 有竞态风险（GIL 在复合操作上不保证原子性）。

·设计思想：Python 的双下划线命名约定是一种"软协议"——不强制、靠自觉。"你千万别用 X 样式命名自己的东西"——这条不成文规矩保护了用户属性与内部字段的隔离。

19.4.4 函数注解与装饰 (Function Annotations and Decorations)

英文原文

For tools roles, functions also support attached annotations—arbitrary user-defined info about a function's arguments and result that augment the function. Python provides syntax for coding annotations, but it do

esn't do anything with them itself—annotations are completely optional, don't impact function behavior in any way, and when present are simply attached to the function object's annotations attribute for use by other tools. While not of general interest to most application programmers, third-party tools and libraries might use annotations in the context of enhanced error checking, or API directives. Type hinting, discussed in Chapter 6, is also based on annotations, but is optional and unused, and doesn't preclude other roles for annotations today (more on this ahead). We studied the formal rules for arguments in function definitions in the preceding chapter. Annotations don't modify these rules but simply extend their syntax to allow extra expressions to be associated with named arguments and function results. Consider the following nonannotated function, coded with three arguments and a returned result:

```
>>> def func(a, b, c): return a + b + c >>> func(1, 2, 3) 6
```

Syntactically, function annotations are coded in `def` header lines (only), as arbitrary expressions associated with arguments and return values. For arguments, they appear after a colon immediately following the argument's name; for return values, they are written after a `->` following the arguments list's closing parenthesis. The following code, for example, annotates all three of the prior function's arguments, as well as its

return value:

```
>>> def func(a:'hack', b: (1, 10), c: float) -> int: return a + b + c
>>> func(1, 2, 3) 6
```

Calls to an annotated function work as usual, but when annotations are present Python collects them in a dictionary and attaches it to the function object itself as its annotations. In this, argument names become keys; the return value annotation is stored under key `return` if coded (which suffices because this reserved word can't be used as an argument name); and the values of annotation keys are assigned to the results of the annotation expressions:

```
>>> func.annotations {'a': 'hack', 'b': (1, 10), 'c': <class 'float'>, 'return': <class 'int'>}
```

Because they are just Python objects attached to a Python object, annotations are straightforward to process. The following annotates just two of three arguments and steps through the attached annotations generically:

```
>>> def func(a:'python', b, c: 3.12): return a + b + c
>>> func(1, 2, 3) 6
```

```
>>> func.annotations {'a': 'python', 'c': 3.12}
```

```
>>> for arg in func.annotations: print(arg, '=>', func.annotations[arg]) a => python
```

`c => 3.12` There are two fine points to note here. First, you can still use defaults for arguments if you code

annotations—the annotation (and its: character) appears before the default (and its = character). In the following, for example, `a:'hack' = 4` means that argument `a` defaults to 4 and is annotated with the string `'hack'`:

```
>>> def func(a:'hack' = 4, b: (1, 10) = 5, c: float = 6)
-> int: return a + b + c >>> func(1, 2, 3) # No defaults used 6
```

```
>>> func() # a=4 + b=5 + c=6 (all defaults) 15
```

```
>>> func(1, c=10) # 1 + b=5 + 10 (keywords work normally) 16
```

```
>>> func.annotations {'a': 'hack', 'b': (1, 10), 'c': <class'float'>, 'return': <class'int'>}
```

Second, note that the blank spaces in the prior example are all optional—you can use spaces between components in function headers or not, but omitting them might degrade your code's readability to some observers (and probably improve it to others):

```
>>> def func(a:'hack'=4, b:(1,10)=5, c:float=6)->int:
return a + b + c >>> func(1, 2) # 1 + 2 + c=6 9
```

```
>>> func.annotations {'a': 'hack', 'b': (1, 10), 'c': <class'float'>, 'return': <class'int'>}
```

While annotations are optional and uncommon, they may be used to specify constraints for types or values, and larger APIs might use this feature as a way to register function interface information. In fact, you'll s

see a potential application in Chapter 39, when we code annotations as an alternative to function decorator arguments—a more general concept in which augmentation info is coded outside the function header and so is not limited to a single role.

中文翻译

出于工具性的用途，函数还支持附加注解（annotations）——关于函数参数和结果的、任意的用户自定义信息，用于增强函数。Python 提供了编写注解的语法，但 Python 自己不会对它们做任何事——注解完全是可选的，不会以任何方式影响函数行为，当存在时，只是被附加到函数对象的 annotations 属性上供其他工具使用。虽然大多数应用程序员对它并不普遍感兴趣，但第三方工具和库可能会在增强的错误检查或 API 指令等场景中使用注解。第 6 章讨论过的类型提示（type hinting）也是基于注解的，但它是可选的、不会被强制使用，而且今天也不排除注解承担其他角色（更多内容见后文）。我们在上一章学习了函数定义中参数的正式规则。注解并不修改这些规则，只是扩展其语法，允许把额外的表达式与命名参数和函数结果关联起来。考虑下面这个无注解的函数，它有三个参数并返回一个结果：

```
python >>> def func(a, b, c): return a + b + c >>> func(1, 2, 3) 6
```

从语法上说，函数注解写在 def 头行（仅限头行）中，是与参数和返回值关联的任意表达式。对于参数，它们出现在紧跟参数名之后的冒号后面；对于返回值，它们写在参数列表右括号之后的 -> 后面。例如，下面的代码为之前那个函数的全部三个参数以及返回值都加了注解：

```
python >>> def func(a:'hack', b: (1, 10), c: float) -> int: r
```

`return a + b + c >>> func(1, 2, 3)` 6 对带注解函数的调用照常工作，但当存在注解时，Python 会把它们收集到一个字典（dictionary）中，作为 annotations 附加到函数对象本身上。在这里，参数名成为键；如果有返回值的注解，则存储在 `return` 键下（这足够了，因为 `return` 这个保留字不能用作参数名）；注解各键的值被赋为注解表达式的求值结果：`python >>> func.annotations {'a':'hack','b': (1, 10),'c': <class'float'>,'return': <class'int'>}` 由于它们只是附加在 Python 对象上的 Python 对象，处理注解非常直接。下面这段代码只为三个参数中的两个加了注解，并通用地遍历附加的注解：`python >>> def func(a:'python', b, c: 3.12): return a + b + c >>> func(1, 2, 3) 6 >>> func.annotations {'a':'python','c': 3.12} >>> for arg in func.annotations: print(arg,'=>', func.annotations[arg]) a => python c => 3.12` 这里有两个细节值得注意。第一，如果你编写注解，仍然可以使用参数的默认值——注解（及其：字符）出现在默认值（及其 = 字符）之前。例如在下面的代码中，`a:'hack' = 4` 意思是：参数 `a` 默认值为 4，并用字符串 `'hack'` 做注解：`python >>> def func(a:'hack' = 4, b: (1, 10) = 5, c: float = 6) -> int: return a + b + c >>> func(1, 2, 3) # 未使用默认值 6 >>> func() # a = 4 + b = 5 + c = 6 (全部使用默认值) 15 >>> func(1, c = 10) # 1 + b = 5 + 10 (关键字参数正常工作) 16 >>> func.annotations {'a':'hack','b': (1, 10),'c': <class'float'>,'return': <class'int'>}` 第二，注意上一个例子中的空格都是可选的——函数头中各组成部分之间你可以加空格也可以不加，但省略它们可能会降低代码对某些人的可读性（对另一些人可能反而更好）：`python >>> def func(a:'hack' =`

```
4, b:(1,10)=5, c:float=6)->int: return a + b + c >>> func(1, 2) # 1 + 2 + c=6 9 >>> func.annotations {'a':'hack','b': (1, 10),'c': <class'float'>,'return': <class'int'>}
```

尽管注解是可选的、也不常见，它们可以用来为类型或值指定约束，而更大的 API 可能会把这个特性当作登记函数接口信息的一种方式。事实上，你将在第 39 章看到一个潜在的应用：我们把注解编码为函数装饰器参数的替代方案——装饰器是一个更通用的概念，其增强信息写在函数头之外，因此不受单一角色的限制。

深度理解

·核心概念：注解 = "函数的便签"。a:'hack'、-> int 只是往函数对象上挂信息（annotations 字典），Python 本身不校验、不执行。类型提示（type hints）是注解最著名的用途，但注解本身是通用的"元数据通道"。

·底层实现：def 语句编译时，注解表达式在定义时就被求值（3.12 里除 from future import annotations 外仍是求值的——注意 c: float 存入的是 <class'float'> 类型对象，c: 3.12 存入的是浮点数 3.12），随后被打包成字典赋给 annotations。运行时没有一次检查——func('a','b','c') 也照常执行。参数默认值语法顺序为 a: annotation = default（先注解后默认值）。

·为什么这样设计：Python 的注解哲学是"语法只管记录，语义交给工具"——让运行时零开销，让类型检查器（mypy、pyright）、API 文档生成器、参数校验框架各取所需。作者特别提醒：注解"几乎没用"（unused），这正是设计者的刻意克制。

·实际问题：现代 Python 中 `def func(x: int, y: str) -> bool` 的注解主要被 IDE 和静态检查器消费，运行时没人读。但注解字典也可被工具读——例如把注解当“契约”实现运行时校验（第 39 章）、生成 REST API 文档。

·初学者误区：① 以为注解会让 Python 检查参数类型——不会，Python 运行时完全忽略它，类型检查要靠 `mypy` 等外部工具；② 混淆注解与默认值——`a: 'hack' = 4` 中 `'hack'` 是注解、`4` 是默认值，顺序不能颠倒；③ 忘了 `return` 键的存在——遍历 annotations 时若只处理参数，会遇到 `'return'` 这个“伪参数”。

·设计思想：`->` 箭头只出现在函数头，这是语法层面的刻意限制——注解必须“一眼能看见、不打扰参数规则”。作者还幽默地指出空格可有可无，但为了人类读者，请保留。

19.4.5 函数装饰器替代方案：预告 (Function Decorators Alternative: Preview)

英文原文

Because we're going to devote an entire chapter to decorators, we'll omit most of their story here. But as a very brief preview, decorators are simply functions that augment other functions. They are applied to another function's `def` with an `@` prefix that rebinds the other function's name to the result of passing it to the decorator. This syntax: `@decorator def func(args): ...`

Decorated function `def` is automatically morphed in to the following equivalent, where `decorator` is a one-argument callable object (or an expression that returns one), which returns a callable object having arguments compatible with `func` (or `func` itself):

```
def func(args): ...func = decorator(func) # Rebind func to decorator's result
```

Decorators can use this hook to wrap another function in an extra layer of code for nearly arbitrary purposes, as in the following code that adds a message when a decorated function is called:

```
def echo(F):
    def proxy(args):
        # Add actions here
        print('calling', F.name)
        return F(args)
    # Run decorated function
    return proxy
@echo
def func(x, y):
    # Rebinds func = echo(func)
    print('I am running...', x, y)
# func is run by the proxy closure
>>> func(1, 2)
# Runs a proxy, which runs func
calling func
I am running... 1 2
```

As you'll learn later, decorators can also take arguments (e.g., `@echo(args)`) whose utility can overlap with annotations—argument info can be sent to the decorator instead of being embedded in headers as annotations. Because this is an advanced tool that can also be applied to classes, we'll pause this thread until Chapters 32 and 39. Compared to annotations, though, decorators don't complicate function headers themselves with extra syntax, and more naturally support multiple roles. Annotations may make functions difficult to read, and generally can have just one role—one that's hijacked by the optional type hinting of Chapter 6 in progra

ms that choose to employ it. In fact, type hinting advocates have tried to deprecate all other roles for annotations. While these divisive (and perhaps even rude) efforts have thankfully failed to date, decorators face no such challenge, and may be a safer bet going forward. Today, though, both annotations and decorators are tools whose roles are limited only by your imagination. Finally, note that annotations and decorators work in `def` statements—but not in lambda expressions, whose syntax by design limits the functions they can define. Coincidentally, this brings us to this potpourri's next topic.

中文翻译

因为我们将用整整一章来讨论装饰器，这里先省略它们的大部分故事。但作为非常简短的预告：装饰器（decorators）就是增强其他函数的函数。它们通过 `@` 前缀应用到另一个函数的 `def` 上，把那个函数的名字重新绑定为“把它传给装饰器”的结果。下面这种语法：python `@decorator` `def func(args): ...` # 被装饰的函数定义会被自动变形为下面这个等价形式，其中 `decorator` 是一个单参数的可调用对象（或返回可调用对象的表达式），它返回一个参数与 `func` 兼容的可调用对象（或返回 `func` 本身）：python `def func(args): ...func = decorator(func)` # 把 `func` 重新绑定为装饰器的结果装饰器可以利用这个钩子把另一个函数包进一层额外的代码，目的几乎不限——就像下面这段代码，在被装饰函数被调用时打印一条消息：python `def echo(F):`
`def proxy(args):` # 在此添加额外动作`print('calling', F.name`

```
me) return F(args) # 运行被装饰的函数
return proxy
@echo
def func(x, y): # 重新绑定: func = echo(func)
    print('I am running...', x, y) # func 由代理闭包运行
>>> func(1, 2) # 运行一个代理, 代理再去运行 func
calling func
I am running... 1 2
```

你以后会学到, 装饰器还可以带参数 (例如 `@echo(args)`), 其用途可能与注解重叠——参数信息可以发给装饰器, 而不是作为注解嵌入函数头。因为这是一个也可以应用于类的高级工具, 我们把这个话题暂停到第 32 章和第 39 章。不过与注解相比, 装饰器不会用额外的语法把函数头本身弄复杂, 而且更自然地支持多个角色。注解可能会让函数难以阅读, 而且通常只能有一个角色——在被选择使用它的程序中, 这个角色还会被第 6 章的可选类型提示“劫持”。事实上, 类型提示的倡导者们曾试图废弃注解的所有其他角色。虽然这些制造分裂 (甚至可以说无礼) 的尝试至今幸未成功, 但装饰器没有这样的挑战, 未来可能是更稳妥的选择。不过在当下, 注解和装饰都是“角色只受想象力限制”的工具。最后请注意: 注解和装饰适用于 `def` 语句——但不适用于 `lambda` 表达式, 后者的语法在设计中就限制了它所能定义的函数。巧合的是, 这正好把我们带到了这锅大杂烩的下一个主题。

深度理解

·核心概念: 装饰器 = “函数加工厂”。`@decorator` 是语法糖, 展开后就是一句普通赋值: `func = decorator(func)`。它把一个函数换成一个“增强版”的函数 (通常是闭包代理)。

·底层实现: `@echo` 下的 `def func(x, y)` 先按普通 `def` 创建函数对象, 随后执行 `func = echo(func)`——名字被重

新绑定。echo 返回的 proxy 是闭包：它捕获了参数 F（原函数），调用 func(1, 2) 时实际上调用 proxy(1, 2) → 打印 calling func → 再调用 F(1, 2)。原函数从未消失，只是被"包"起来了。

·为什么这样设计：作者强调装饰器 vs 注解的对比：注解的信息"内嵌"在函数头（-> int 挤在签名里），装饰器的信息"外包"（@echo 独立一行）——后者不污染签名、可以叠加多个、角色不受限。这背后是"关注点分离"思想：横切关注点（日志、计时、鉴权）不该混进业务签名。

·实际问题：@staticmethod、@property、@functools.lru_cache、Flask 路由 @app.route(...) 都是装饰器。日志、重试、缓存、权限校验是装饰器的四大经典用途。学习本章只需"看懂"，实现细节在第 32/39 章。

·初学者误区：① 以为 @echo 是"声明"——它实际执行了 echo(func) 并把返回值重新绑定；② 忘了装饰器函数本身也是函数，其返回值必须是可调用对象；③ 误以为注解和装饰器互斥——它们可以同时使用，只是角色不同。

·设计思想：作者对"类型提示倡导者想废除注解其他用法"的评论带点幽默的立场声明——注解的未来有争议，而装饰器地位稳固。这提醒读者：工具选型要考虑"生态共识"。

19.5 匿名函数：lambda (Anonymous Functions: lambda)

英文原文

We first met the lambda expression back in Chapter 16 and have used it in a handful of examples since then. This section reviews the basics and takes a deeper second look, to both reinforce and expand on this topic. As we've seen, besides the `def` statement, Python provides an expression that creates function objects. Because of its similarity to a tool in other languages, it's called `lambda`. Like `def`, this expression creates a function to be called later, but it returns the function instead of assigning it to a name. This is why `lambdas` are sometimes known as anonymous functions. In practice, they are used to inline function definitions, or defer execution of code. NOTE: What's in a name? The `lambda` tends to intimidate people, largely due to the name "lambda" itself—a name that comes from the Lisp language, which got it from lambda calculus, which is a form of symbolic logic. Obscure mathematical heritage aside, though, `lambda` in Python is really just a word that introduces an expression syntactically, and its expression is simply an alternative way to code a function—albeit without statements, decorators, or annotations.

中文翻译

我们最早在第 16 章见过 `lambda` 表达式，此后又在好几个例子中使用过它。本节将复习基础知识，并进行更深入的第

二次审视，既巩固也扩展这个话题。正如我们所见，除了 `def` 语句之外，Python 还提供了一个创建函数对象的表达式。由于它与其他语言中一个工具相似，所以被称为 `lambda`。与 `def` 一样，这个表达式会创建一个稍后调用的函数，但它返回函数而不是把它赋值给一个名字。这就是 `lambda` 有时被称为匿名函数 (anonymous functions) 的原因。在实践中，它们用于内联 (inline) 函数定义，或延迟 (defer) 代码的执行。附注：名字里有什么？`lambda` 往往让人望而生畏，很大程度上是因为“`lambda`”这个名字本身——这个名字来自 Lisp 语言，而 Lisp 又是从 `lambda` 演算 (`lambda calculus`，一种符号逻辑) 那里得来的。抛开晦涩的数学血统不谈，Python 中的 `lambda` 其实只是一个在语法上引入表达式的词，它的表达式只是编写函数的另一种方式——只是没有语句、装饰器或注解。

深度理解

·核心概念：`lambda` 与 `def` 的本质区别只有两条：① `lambda` 是表达式，`def` 是语句——表达式有值、可出现在任何“值”的位置（列表里、调用参数里），语句则不行；② `lambda` 体只能是一个表达式，不能是语句块——没有 `if` 块、没有 `for`、没有显式 `return`。

·底层实现：`lambda` 表达式的执行就是“创建一个函数对象并返回它”。CPython 中 `lambda x: x+1` 编译出的字节码与 `def f(x): return x+1` 的主体完全相同（都是 `LOADFAST x; LOADCONST 1; BINARYOP +; RETURNVALUE`），差别只在于：`def` 的名字绑定是编译期安排的赋值，`lambda` 的名字绑定（如果有）由外层表达式负责。`lambda`

da 对象同样有 name (固定为 <lambda>)、annotations (空的) 等属性。

·为什么这样设计：名字 "lambda" 的历史： λ 演算 $\rightarrow$ Lisp $\rightarrow$ Python。作者特意"祛魅"：名字吓人，概念不吓人。Python 设计 lambda 为表达式，是为了在"语句不被允许"的地方（列表字面量、函数调用参数、字典值）内嵌小函数。

·实际问题：GUI 回调 `command=lambda: ...`、`sorted(iterable, key=lambda x: x[1])`、`map/filter` 的处理函数——都是 lambda 的主场。

·初学者误区：① 怕 lambda——它只是"没有名字的 def"，勇气比技巧重要；② 试图在 lambda 里写 `if ...` 语句或 `return`——语法错误，需要改用三元表达式；③ 以为 lambda 一定比 def 快/省内存——不，两者产物等价，性能无差别。

19.5.1 lambda 基础 (lambda Basics)

英文原文

As a refresher, the lambda's general form is the keyword `lambda`, followed by zero or more arguments (just like the arguments you enclose in parentheses in a `def` header, sans annotations), followed by an expression after a colon:

```
lambda argument1, argument2,... argumentN: expression-using-arguments
```

Parentheses are not allowed around lambda arguments and are generally optional around the entire lambda itself, though they're required in some contexts and may boost clarity in others. Functions returned by lambda work the same as those assigned by def, but lambda differs in ways that make it useful in specialized roles: lambda is an expression, not a statement. Because of this, a lambda can appear in places a def is not allowed by Python's syntax—inside a list literal or a function call's arguments, for example. With def, functions can be referenced by name in such locations, but must be created elsewhere. As an expression, lambda returns a value (a new function) that can be assigned a name, or used where the lambda appears. lambda's body is a single expression, not a block of statements. The lambda's body is similar to what you'd put in a def body's return statement; you simply type the result as a naked expression, instead of explicitly returning it. Because it is limited to an expression, a lambda is less general than a def—you can only squeeze so much logic into a lambda body without full-blown statements. This is by design, to limit program nesting: lambda is designed for coding simple functions, and def handles larger tasks. Apart from those distinctions, defs and lambdas do the same sort of work. For instance, by this point we should be pros at making a function with a def statement:

```
>>> def func(x, y, z): return x + y + z >>> func(2, 3, 4) 9
```

But we can achieve the same effect with a lambda expression by explicitly assigning its result to a name through which you can later call the otherwise-anonymous function:

```
>>> func = lambda x, y, z: x + y + z >>> func(2, 3, 4) 9
```

Here, `func` is manually assigned the function object the lambda expression creates; this is how `def` works, too, but its assignment is automatic. Defaults and other argument-matching syntax work in lambda too, just like in a `def`:

```
>>> x = (lambda a='hack', b='python', c='code': a + b + c) >>> x('write')'writepythoncode'
```

The code in a lambda body also follows the same scope lookup rules as code inside a `def`. lambda expressions introduce a new local scope much like a nested `def`, which automatically sees names in enclosing functions, the module, and the built-in scope—via the LEGB rule we studied in Chapter 17:

```
>>> def editions(title): # title in enclosing scope return (lambda e: title + ',' + e) # Return a function object >>> labeler = editions('Learning Python') # Make+save nested function >>> labeler('6E') #'6E' passed to e in lambda 'Learning Python, 6E'
```

With those basic lambda "hows" in hand, the natural next question is "why," taken up in the next section.

中文翻译

作为复习，lambda 的一般形式是关键字 lambda，后面跟着零个或多个参数（就像你在 def 头中放在括号里的参数，只是没有注解），再后面是冒号和表达式：lambda 参数1, 参数2,...参数N：使用参数的表达式参数两边不允许加括号，而整个 lambda 外面的括号通常是可选的——不过在某些上下文中它们是必需的，在其他上下文中也许能提高清晰度。lambda 返回的函数与 def 赋值的函数工作方式相同，但 lambda 的不同之处使它适用于专门的角色：lambda 是表达式，不是语句。正因如此，lambda 可以出现在 Python 语法不允许 def 出现的地方——例如列表字面量内部或函数调用的参数列表里。使用 def 时，函数可以在这样的位置按名字引用，但必须在他处创建。作为表达式，lambda 返回一个值（一个新函数），这个值可以赋给名字，也可以在 lambda 出现的位置直接使用。lambda 的函数体是单个表达式，而不是语句块。lambda 的函数体类似于你会放进 def 函数体 return 语句里的内容；你只是把结果作为裸表达式打出来，而不是显式返回它。因为受限于单个表达式，lambda 不如 def 通用——没有完整的语句，你只能在 lambda 体里塞下有限的逻辑。这是有意为之，以限制程序嵌套：lambda 为编写简单函数而设计，def 处理更大的任务。除了这些区别，def 和 lambda 做的是同类工作。例如，到目前我们应该是用 def 语句创建函数的高手了：python >>> def func(x, y, z): return x + y + z >>> func(2, 3, 4) 9 但我们也可以用 lambda 表达式达到同

样效果——只要显式地把它的结果赋给一个名字，之后就能通过这个名字调用这个本来匿名的函数：`python >>> func = lambda x, y, z: x + y + z >>> func(2, 3, 4)` 9 这里，`func` 被手动赋值为 `lambda` 表达式创建的函数对象；`def` 也是这么工作的，只是它的赋值是自动的。默认值和其他参数匹配语法同样适用于 `lambda`，就像在 `def` 中一样：`python >>> x = (lambda a='hack', b='python', c='code': a + b + c) >>> x('write')'writepythoncode'` `lambda` 函数体内的代码也遵循与 `def` 内部代码相同的作用域查找规则。`lambda` 表达式会引入一个新的局部作用域，很像嵌套的 `def`，它自动“看得见”包围它的函数、模块以及内置作用域中的名字——也就是我们在第 17 章学过的 LEGB 规则：`python >>> def editions(title): # title 位于包围它的作用域 return (lambda e: title + ',' + e) # 返回一个函数对象 >>> labeler = editions('Learning Python') # 创建并保存嵌套函数 >>> labeler('6E') # '6E' 传给 lambda 中的 e 'Learning Python, 6E'` 掌握了这些基本的 `lambda` “怎么用”之后，自然而然的下一问就是“为什么用”，下一节将予以解答。

深度理解

·核心概念：`lambda` 的语法解剖：`lambda` 参数：表达式。三个要点：① 参数列表不带括号（与 `def` 不同！）；② 冒号后是裸表达式（隐式 `return`）；③ 整体是表达式，可赋值、可传参、可内嵌。

·底层实现：`func = lambda x, y, z: x + y + z` 编译后，`func` 名字绑定到一个函数对象——它的字节码与 `def func(x,y,z): return x+y+z` 逐字节相同。`x = (lambda a='hac`

k', ...) 外层括号只是把 lambda 与 x(...) 调用分隔开，让解析器不混淆。editions 的例子中，lambda 捕获 title 形成闭包（单元格变量，LOADDEREF 读取）。

·为什么这样设计：为什么参数不能加括号？——避免与"调用表达式"语法歧义，保持 lambda 的极简形式。为什么体只能是表达式？——表达式能出现在任何值的位置，语句不能；要"内联"，就必须放弃语句。这是用表达力换取位置自由度的设计交易。

·实际问题：LEGB 规则对 lambda 同样成立——这正是 GUI 回调能"看见"外层 self、能记住循环变量前当前值的原因（第 17 章提过循环变量陷阱：lambda 延迟执行时捕获的是最终值，除非用默认参数绑定快照）。

·初学者误区：① 给 lambda 参数加括号 lambda (x, y): ..——语法错误，这是旧版 Python 2 的写法；② 以为 lambda 不能有默认参数/args——能，规则与 def 相同；③ 把 editions 里的 lambda 想成"字符串拼接"——它是"返回了一个函数，函数被调用时才拼接"，labeler('6E') 才真正执行拼接。

19.5.2 为什么使用 lambda? (Why Use lambda?)

英文原文

Generally speaking, lambda is a sort of function shorthand that allows you to embed a function's definition within the code that uses it. It is entirely optional—

you can always use `def` instead, and should if your function requires the power of full statements that the lambda's expression cannot provide. But lambda may be easier to use in scenarios where you just need to embed a small bit of executable code inline at the place where it is to be later used. For instance, you'll see later that callback handlers are frequently coded as inline lambda expressions embedded directly in a registration call's arguments list, instead of being defined with a `def` elsewhere in a file and referenced by name (see the sidebar "Why You Will Care: lambda Callbacks" for an example). lambda is also commonly used to code jump tables, which are lists or dictionaries of actions to be performed on demand. For example:

```
L = [
    lambda x: x * 2, # Inline function definition
    lambda x: x * 2,
    lambda x: x // 2] # A list of three callable functions
for f in L: print(f(5)) # Prints 10, 25, and 2
print(L[1]) # Prints 25
```

The lambda expression works well when you need to stuff small pieces of executable code into places where statements like `def` are not allowed. The preceding code snippet, for example, builds up a list of three functions by embedding lambda expressions inside a list literal; a `def` won't work inside a literal like this because it is a statement, not an expression. The equivalent `def` coding would require temporary function names (which might clash with others) and function definitions outside the context of intended use (which might be hundreds of lines away):

```
def f1(x): ret
```

```
urn x 2 def f2(x): return x 2 # Define named functions
def f3(x): return x // 4 # Separate from their place of
use L = [f1, f2, f3] # Reference by name for f in L: prin
t(f(2)) # Also prints 10, 25, and 2
```

中文翻译

一般而言，lambda 是一种函数简写 (shorthand)，让你能把函数的定义嵌入到使用它的代码中。它完全是可选的——你总是可以用 def 代替，而且如果你的函数需要完整语句的力量 (lambda 的表达式提供不了)，你就应该用 def。但在某些场景下，你只需要在“将来要用的地方”内联一小段可执行代码，此时 lambda 可能更顺手。例如，你稍后会看到，回调处理器 (callback handlers) 经常被写成内联 lambda 表达式，直接嵌入注册调用的参数列表里，而不是用 def 在文件他处定义、再按名字引用 (示例见侧边栏《你会在意的原因：lambda 回调》)。lambda 也常被用来编写跳转表 (jump tables) ——即按需执行的动作列表或字典。例如：python L = [lambda x: x 2, # 内联函数定义 lambda x: x 2, lambda x: x // 2] # 三个可调用函数的列表 for f in L: print(f(5)) # 打印 10、25 和 2 print(L1) # 打印 25 当你需要把一小段可执行代码塞进“不允许 def 语句”的地方时，lambda 表达式表现优异。例如，前面这段代码通过在列表字面量内部嵌入 lambda 表达式，构建了一个由三个函数组成的列表；def 无法在这样的字面量内部工作，因为它是语句而不是表达式。等价的 def 编码需要临时的函数名 (可能与其他名字冲突)，以及离开使用上下文 (可能相距数百行) 的函数定义：python def f1(x): return x 2 def f2(x): return x 2 # 定义有名字的函数 def f3

```
(x): return x // 4 # 与使用位置分离 L = [f1, f2, f3] # 按名字引用
for f in L: print(f(2)) # 同样打印 10、25 和 2
```

深度理解

·核心概念：lambda 的最大卖点 = 代码邻近性 (code proximity) —— 定义与使用在同一处，语义一目了然。跳转表 (jump table) 是经典用法：把"动作"组织成数据 (列表/字典)，再统一按索引/键触发。

·底层实现：L = [lambda x: x², ...] 执行时，三个 lambda 表达式各自创建独立的函数对象存入列表。for f in L: print(f(5)) 只是逐个取出函数并调用。注意：列表构建时函数体内代码尚未执行——x² 要等 f(5) 调用时才运行，这就是"延迟执行"。

·为什么这样设计：为什么不用 if/elif 链？——对于"根据状态选动作"的需求，if 链是命令式思维，跳转表是数据驱动思维：动作以数据形态存在，可以动态增删、可以整体传递。函数即对象让"行为"与"数据"统一了形态。

·实际问题：命令处理器 (CLI 子命令映射)、事件分发器、策略模式 (可替换的算法集合)、sorted(key=...)——全是跳转表思想的变体。GUI 回调注册 Button(command=lambda: ...) 是 lambda 最常见的日常角色。

·初学者误区：① 以为 L 里存的是"计算结果"——不，存的是函数，f(5) 才算结果；② 为了几个小函数专门 def f1/f2/f3 制造命名负担和冲突风险——这正是 lambda 存在的原因，但注意 def 版的示例里 f3 写的是 x // 4，与原 lambda 的 x // 2 不同，作者的示例本身打印结果就依赖

这个差异 (f(2) 时 $2//4=0$实际输出会是 4, 4, 0, 作者在前一行注释里写的 "Also prints 10, 25, and 2" 严格说只对 f(2) 的原列表成立, 这里无需深究, 重点是"名字分散、代码割裂"的缺点) ; ③ 忽视"延迟执行"——lambda 内的代码在定义时不运行, 只在调用时运行, 这是回调机制能工作的前提。

19.5.3 多路分支: 终章 (Multiway Branches: The Finale)

英文原文

In fact, you can do the same sort of thing with dictionaries and other data structures in Python to build up more general sorts of action tables. Here's another example to illustrate at the interactive prompt:

```
>>> key = 'loop'>>> {'hack': lambda s: s.upper(), 'code': lambda s: s.lower(), 'loop': lambda s: f'{s 4}!'}key'PyPyPyPy!'
```

Here, when Python makes the temporary dictionary, each of the nested lambdas generates and leaves behind a function to be called later. Indexing by key fetches one of those functions, and parentheses force the fetched function to be called. When coded this way, a dictionary becomes a more general multiway branching tool than this book could fully reveal in Chapter 12's coverage of if statements. To make this work without lambda, you'd need to instead code three de

f statements somewhere else in your file, outside the dictionary in which the functions are to be used, and reference the functions by name:

```
>>> def f1(s): return s.upper() >>> def f2(s): return  
s.lower() >>> def f3(s): return f'{s 4}!'>>> key = 'loop'  
>>> {'hack': f1,'code': f2,'loop': f3}key'PyPyPyPy!'
```

This works, too, but your defs may be arbitrarily far a way in your file, even if they are just little bits of code. The code proximity that lambda provides is especially useful for functions that will only be used in a single context—if the three functions here are not useful anywhere else, it makes sense to embed their definitions within the dictionary as lambdas. Moreover, the def form requires you to make up names for these little functions that, again, may clash with other names in this file (perhaps unlikely, but always possible). You can use the match statement today with similar results, but it may require even more code than the def scheme, especially since you'd have to nest it within a def to support an argument like s; see Chapter 12 for more info. lambda is also handy in function-call argument lists as a way to inline temporary function definitions not used anywhere else in your program; you'll see examples of such other uses later in this chapter, when we study map. NOTE: The siren song of eval. In principle, you could skip the dispatch table in the preceding code if the function name is the same as its string lookup key—an eval(key)(arg) would kick off

the call. While true in this case and sometimes useful, as we saw earlier (e.g., Chapter 10), eval is relatively slow (it must compile and run code) and insecure (you must trust the string's source). More fundamentally, jump tables are generally subsumed by polymorphic method dispatch in Python: calling a method does the "right thing" based on the type of object that's the subject of the call, no switching logic required. To see why, stay tuned for Part VI.

中文翻译

事实上，在 Python 中你可以用字典和其他数据结构做同样的事，构建出更通用的动作表（action tables）。下面是另一个在交互式提示符下演示的例子：`python >>> key = 'loop'>>> {'hack': lambda s: s.upper(),'code': lambda s: s.lower(),'loop': lambda s: f'{s 4}!'}key'PyPyPyPy!`在这里，当 Python 创建这个临时字典时，每个嵌套的 lambda 都会生成并留下一个供稍后调用的函数。用键索引取回其中一个函数，再用括号强制调用取回的函数。这样编码时，字典成为比第 12 章 if 语句所能展示的更通用的多路分支（multiway branching）工具。不用 lambda 要实现这一点，你需要改为在文件某处——在要使用这些函数的字典之外——编写三条 def 语句，并按名字引用函数：`python >>> def f1(s): return s.upper() >>> def f2(s): return s.lower() >>> def f3(s): return f'{s 4}!'>>> key = 'loop'>>> {'hack': f1,'code': f2,'loop': f3}key'PyPyPyPy!`这样也行，但你的 def 可能位于文件中任意远的地方，即使它们只是几小段代码。lambda 提供的代码邻近性对“只会在

单一上下文中使用"的函数尤其有用——如果这里的三个函数在其他任何地方都用不到，把它们的定义作为 lambda 嵌入字典里是合理的。此外，def 形式要求你为这些小函数编造名字，而这些名字同样可能与本文件中的其他名字冲突（也许不太可能，但总是有可能）。你今天也可以使用 match 语句获得类似结果，但它可能比 def 方案需要更多代码，尤其是你必须把它嵌套在 def 里才能支持像 s 这样的参数；更多信息见第 12 章。lambda 在函数调用的参数列表里也很方便，可以用来内联"程序其他任何地方都不会用到"的临时函数定义；本章稍后学习 map 时，你会看到这类其他用法的例子。附注：eval 的塞王之歌。原则上，如果函数名与它的字符串查找键相同，你可以跳过前面代码里的分发表——eval(key)(arg) 就能启动调用。虽然在本例中确实如此、有时也有用，但正如我们前面所见（例如第 10 章），eval 相对较慢（它必须编译并运行代码）且不安全（你必须信任字符串的来源）。更根本的是，跳转表在 Python 中通常被多态方法分发（polymorphic method dispatch）所取代：调用一个方法时，会根据调用主体（subject）对象的类型自动做"正确的事"，不需要任何切换逻辑。想明白为什么，敬请期待第六部分。

深度理解

·核心概念：字典即函数 = 通用多路分支。{键: 函数}键一次性完成"查表 + 调用"，这是 switch/case 语句的 Python 惯用法。键可以是字符串、整数、元组——比 if/elif 更灵活（可动态增删条目）。

·底层实现：执行顺序：① 构建临时字典（三个 lambda 各创建函数对象）；② [key] 触发字典哈希查找（getite

m) , O(1) 取回函数; ③ (...) 立即调用它。三步连写的 {...}key 对新手是一堵语法墙, 拆开就是 d = {...}; f = d[key]; f('Py')。

·为什么这样设计: 作者特意提醒 eval(key)(arg) 的诱惑并劝退——eval 把字符串当代码执行, 慢(要经过编译)且险(字符串不可信 = 任意代码执行)。这引出一条重要设计哲学: 数据驱动的表查找优于字符串驱动的动态求值。

·实际问题: CLI 命令分发、HTTP 路由、状态机迁移表、插件注册表——都是"键 → 动作"映射。与 match 对比: match 是结构化的静态分支, 字典是数据化的动态分支; 动态意味着可以在运行时注册新分支(插件机制)。

·初学者误区: ① 看不懂 {'...': lambda}key 的连续语法——把它逐步拆开; ② 想用 eval 走捷径——安全红线, 永远不要对不可信字符串执行 eval; ③ 以为多路分支必须用 if/elif——字典查表通常更快、更易维护(键即文档)。

·设计思想: 作者把"多态分发取代跳转表"留到第六部分——预告 OOP 让"切换逻辑"消失: obj.action() 根据 obj 的类型自动路由, 无需手写分支。这是"开闭原则"(对扩展开放、对修改关闭)的 Python 化表达。

19.5.4 如何(不)让你的 Python 代码难以阅读 (How (Not) to Obfuscate Your Python Code)

英文原文

The fact that the body of a lambda has to be a single expression (not a series of statements) would seem to place severe limits on how much logic you can pack into a lambda. If you know what you're doing, though, you can code most statements in Python as expression-based equivalents.

For example, if you want to print from the body of a lambda function, simply use `print` or `sys.stdout.write` (recall from Chapter 11 that this is what `print` really does). And to execute multiple actions, code a sequence like a tuple, to evaluate nested multiple expressions from left to right (tuples require parentheses in this context):

```
>> series = lambda a, b: (print(a.upper()), print(b.lower())) # > 1 actions
>> ignore = series('Hack','Code')
HACK code
```

Similarly, to nest logic in a lambda, you can use the `if/else` ternary expression introduced in Chapter 12, or the equivalent but trickier and/or combination also described there. As you learned earlier, the following statement:

```
if a: b
else: c
```

can be emulated by either of these equivalent expressions (the second is only roughly the same, but clos

e enough):

b if a else c ((a and b) or c)

Because expressions like these can be placed inside a lambda, they may be used to implement selection logic within a lambda function (the lambda is parenthesized here only for variety and subjective clarity):

```
>> lower = (lambda x, y: x if x < y else y) # Ifs in lambda: ternary
>> lower('bb','aa')
'aa'
>> lower('aa','bb')
'bb'
```

Furthermore, if you need to perform loops within a lambda, you can also embed things like list comprehension expressions and map calls—tools we met in earlier chapters and will revisit in this and the next chapter:

```
>> showall = lambda x: [print(y) for y in x] # Loops in lambda: list comp
>> t = showall(['lp5e','lp6e'])
lp5e
lp6e
>> showall = lambda x: list(map(print, x)) # Loops in lambda: maps
>> t = showall(['py3.3','py3.12'])
py3.3
py3.12
>> showall = lambda x: print(x, sep='', end='') # Another option: unpacking
```

And while it's limited by the local scope of the function that lambda makes, assignment is in scope (pun intended) for lambda expressions that use the := named-assignment expression:

```
>> namer = lambda x: (res := x + 1) + res # Assignment in lambda: :=
>> namer(2)
6
>> res
6
```

't persist `NameError: name 'res' is not defined`

There is a limit to emulating statements with expressions: you can't assign nonlocals, for instance, though tools like the `setattr` built-in, the dict of namespaces, and methods that change mutable objects in place can sometimes stand in—and can quickly lead you deep into the heart of expression-convolution darkness.

But now that this book has shown you these tricks, it must also humbly implore you to use them only as a last resort. Without due care, they can yield unreadable (a.k.a. obfuscated) Python code, despite the language's best intents. In general, simple is better than complex, explicit is better than implicit, and full statements are better than arcane expressions. That's why `lambda` is limited to expressions.

If you have larger logic to code, use `def`; `lambda` is for small pieces of inline code. On the other hand, you may find these techniques useful—when taken in moderation.

中文翻译

`lambda` 的函数体必须是单个表达式（而不是一系列语句），这一事实似乎严重限制了你能往 `lambda` 里塞多少逻辑。但如果你知道自己在做什么，Python 中大多数语句都可以写成基于表达式的等价形式。例如，如果要从 `lambda` 函数体内打印，直接用 `print` 或 `sys.stdout.write` 即可（回忆第 11 章：这就是 `print` 真正做的事）。而要执行多个动

作，可以编码一个像元组那样的序列，从左到右依次求值嵌套的多个表达式（元组在这个上下文中需要括号）：`python >>> series = lambda a, b: (print(a.upper()), print(b.lower()))` # 超过 1 个动作

`>>> ignore = series('Hack','Code')` HACK code 类似地，要在 lambda 中嵌套逻辑，可以使用第 12 章介绍的 if/else 三元表达式，或者第 12 章同样介绍过的、等价但更棘手的 and/or 组合。你之前学过，下面这条语句：`python if a: b else: c` 可以用下面两个等价表达式之一来模拟（第二个只是大致相同，但也足够接近）：`python b if a else c ((a and b) or c)` 因为这类表达式可以放进 lambda 内部，它们可以用来在 lambda 函数内实现选择逻辑（这里的 lambda 加括号只是为了换风格 and 主观上的清晰）：`python >>> lower = (lambda x, y: x if x < y else y)` # lambda 里的 if: 三元表达式

`>>> lower('bb','aa')'aa'` `>>> lower('aa','bb')'aa'` 此外，如果你需要在 lambda 内执行循环，你还可以嵌入列表推导式表达式和 map 调用之类的东西——这些工具我们在前面章节遇到过，并将在本章和下一章重新审视：`python >>> showall = lambda x: [print(y) for y in x]` # lambda 里的循环：列表推导式

`>>> t = showall(['lp5e','lp6e'])` lp5e lp6e

`>>> showall = lambda x: list(map(print, x))` # lambda 里的循环：map

`>>> t = showall(('py3.3','py3.12'))` py3.3 py3.12

`>>> showall = lambda x: print(x, sep="", end=")` # 另一种方案：解包而且，尽管赋值受限於 lambda 所创建函数的局部作用域，对于使用 `:=` 命名赋值表达式（named-assignment expression）的 lambda，赋值（assignment）是在作用域内（双关语）的：`python >>> namer = lambda x: (res := x + 1) + res` # lambda 里的赋

值：`:= >>> namer(2) 6 >>> res` # 但它不会留存 `NameError: name 'res' is not defined` 用表达式模拟语句是有极限的：例如，你不能给 `nonlocal` 赋值，尽管 `setattr` 内置函数、命名空间的 `dict`、以及就地修改可变对象的方法有时可以顶替——但这些手段很快就会把你带进“表达式纠结黑暗”的深渊中心。但既然这本书已经向你展示了这些技巧，它也必须谦卑地恳求你：只把它们作为最后手段。若不慎，它们会产出不可读的（又称混淆（obfuscated）的）Python 代码——尽管语言本意并非如此。总的来说，简单优于复杂，显式优于隐式，完整语句优于晦涩表达式。这正是 `lambda` 被限制为表达式的原因。如果你有更大的逻辑要写，请用 `def`；`lambda` 是给小块内联代码用的。另一方面，你也许会觉得这些技巧有用——只要适度使用。

深度理解

·核心概念：`lambda` 的“作弊大全”：元组序列（多动作）、三元表达式（分支）、列表推导式/`map`（循环）、`:=` 海象运算符（赋值）。作者演示这些，目的是劝退——“你可以，但别这么做”。

·底层实现：`(print(a.upper()), print(b.lower()))` 是元组构造——两个 `print` 从左到右求值，元组本身被丢弃（赋给 `ignore`）。`[print(y) for y in x]` 是列表推导式——副作用是打印，列表本身被丢弃。`(res := x + 1) + res` 中海象运算符把值 3 赋给 `res` 并返回 3，于是 `3 + 3 = 6`；但 `res` 是 `lambda` 的局部变量，退出后消失——外面 `res` 未定义报 `NameError`。

·为什么这样设计：Python 的表达式体系足够完整，几乎

能用表达式模拟一切语句——这是语言"一致性"的副产品。但作者用整节篇幅的潜台词是：能力不等于应该。"Simple is better than complex"（简单优于复杂）正是《Python 之禅》的原话。

·实际问题：什么时候这些技巧真有用？——极少数"必须在表达式位置完成小逻辑"的场景（如 `sorted(key=lambda x: ...)` 内部）。日常开发中，多行 `def` 永远更清晰。代码评审时看到花式 `lambda`，应该警惕。

·初学者误区：① 学会一个技巧就到处用——作者明确说 "use them only as a last resort"；② 误解 `(a and b) or c` 的短路语义——它只在 `b` 为真值时等价于三元表达式，`b` 为假（0、""、None）时结果会错；③ 以为 `:=` 定义的变量会泄漏到全局——不会，它遵守 LEGB，只存在于 `lambda` 的局部作用域（这点和循环中的 `:=` 不同，循环里它属于外围作用域）。

19.5.5 作用域：lambda 也可以嵌套 (Scopes: Lambdas Can Be Nested Too)

英文原文

One last lambda note: as we saw in Chapter 17, lambda is also the main beneficiary of enclosing-function scope lookup—the E in the LEGB scope rule. As a review, in the following the lambda appears inside a def, its typical coding, and so can access the value that name `x` had in the enclosing function's scope during its

call: >>> def action(x): return (lambda y: x + y) # Make function, remember x
>>> act = action(99) >>> act(2) # Call what action returned 101 What wasn't illustrated in the prior discussion of nested function scopes is that a lambda also has access to the names in any enclosing lambda. This case is somewhat obscure, but imagine if we recoded the prior def with a lambda:
>>> action = (lambda x: (lambda y: x + y)) # lambda scopes nest too
>>> act = action(99) >>> act(3) 102
>>> ((lambda x: (lambda y: x + y))(99))(4) # Even if you don't save them 103
Here, the nested lambda structure makes a function that makes a function when called. In both cases, the nested lambda's code has access to the variable x in the enclosing lambda. This works, but it seems fairly convoluted code; in the interest of readability, nested lambdas may be best avoided. The good news, perhaps, is that def cannot be nested in lambda: because lambda's body is an expression, statements like def won't work—thankfully!

中文翻译

关于 lambda 的最后一个说明：正如我们在第 17 章看到的，lambda 也是包围函数作用域查找（LEGB 规则中的 E）的主要受益者。作为复习，在下面这段代码中，lambda 出现在 def 内部——这是它的典型写法——因此能访问名字 x 在包围函数作用域中调用期间的值：python
>>> def action(x): return (lambda y: x + y) # 创建函数，记住 x
>>> act = action(99) >>> act(2) # 调用 action 返回的东

西101 之前关于嵌套函数作用域的讨论中没有说明的是：lambda 也能访问任何包围它的 lambda 中的名字。这种情况有点晦涩，但想象一下如果我们把前面的 def 改写成 lambda：python >>> action = (lambda x: (lambda y: x + y)) # lambda 作用域同样可以嵌套>>> act = action(99) >>> act(3) 102 >>> ((lambda x: (lambda y: x + y))(99))(4) # 即使你不保存它们103 在这里，嵌套的 lambda 结构制造了一个“被调用时制造函数”的函数。两种情况下，嵌套 lambda 的代码都能访问外层 lambda 中的变量 x。这样能工作，但看起来相当迂回；出于可读性的考虑，嵌套 lambda 最好避免。也许好消息是：def 不能嵌套在 lambda 中——因为 lambda 的函数体是表达式，像 def 这样的语句无法工作——谢天谢地！

深度理解

·核心概念：闭包机制对 def 和 lambda 一视同仁——lambda 嵌套在 def 里（常见）、lambda 嵌套在 lambda 里（不常见但合法），内层都能捕获外层作用域的变量。LEGB 规则对表达式创建的函数同样生效。

·底层实现：action = (lambda x: (lambda y: x + y)) 创建外层函数；action(99) 调用它，返回内层 lambda——内层 lambda 的闭包单元格捕获了 x=99；act(3) 调用内层， $x + y = 99 + 3 = 102$ 。((lambda x: (lambda y: x + y))(99))(4) 一步完成同样流程：创建→调用→返回→调用，中间结果不落名。字节码层面：内层函数对象创建时，解释器记录它引用的自由变量（free variable），运行时用 LOADDEREF 从单元格读取。

·为什么这样设计：因为闭包（closure）是"函数 + 被捕获的环境"的统一机制，语言层面没有理由对 def 和 lambda 区别对待。嵌套 lambda 能工作，是设计一致性的自然结果，而非刻意功能。

·实际问题：实践中基本用不到 lambda 套 lambda——functools.partial 或普通 def 更清晰。但理解"任何函数都能捕获外层变量"对调试闭包相关 bug 很重要（例如回调里"所有按钮都打印最后一个值"的经典陷阱）。

·初学者误区：① 以为"lambda 作用域小，看不到外层 lambda 的变量"——看得见，LEGB 一路向上；② 把 (lambda x: (lambda y: x+y)) 当"两个参数的函数"——它是"单参数函数返回单参数函数"，柯里化（currying）思想；③ 想在 lambda 体里再写 def——语法错误，def 是语句不是表达式。作者最后一句"thankfully!"（谢天谢地）是教学幽默：语言设计故意不让 lambda 无限嵌套，保住了可读性底线。

19.6 函数式编程工具 (Functional Programming Tools)

英文原文

Last up in this chapter's gumbo is a reprisal of a category of tools we met in earlier in this book, with a few new tips to round out the topic. By most definitions, today's Python blends support for multiple programming paradigms: procedural (with its basic stateme

nts), object-oriented (with its classes), and functional. The criteria for the latter of these categories are somewhat loose, but by most measures Python's functional programming toolbox includes its first-class object model, nested scope closures, and anonymous function lambdas that we met earlier; its generators and comprehensions, which we'll be expanding on in the next chapter; and perhaps its function and class decorators previewed here but fleshed out in this book's final part. This toolbox also includes built-in functions that apply other functions to iterables automatically—including functions that call other functions on an iterable's items (map); select items based on a test function (filter); and apply functions to pairs of items and running results (reduce). Let's wrap up this chapter with a quick survey of this trio.

中文翻译

本章这锅“秋葵浓汤”的最后一道菜，是重新回顾本书早些时候遇到过的一类工具，并补充几个新技巧来完善这个话题。按大多数定义，今天的 Python 融合了对多种编程范式的支持：过程式（procedural，以其基本语句为核心）、面向对象（object-oriented，以其类为核心）、以及函数式（functional）。后一个类别的标准有些宽松，但按大多数衡量标准，Python 的函数式编程工具箱包括：我们前面遇到的一等对象模型、嵌套作用域闭包、匿名函数 lambda；生成器（generators）和推导式（comprehensions）——下一章将深入展开；或许还有这里预告过、但在本书最后部

分完整展开的函数装饰器和类装饰器。这个工具箱还包括一些内置函数，它们自动把其他函数应用于可迭代对象（iterables）——包括在可迭代对象的每个项上调用其他函数的 map；基于测试函数选择项（select items）的 filter；以及把函数应用于"项对 + 运行中的累计结果"的 reduce。让我们以对这三件套的快速巡视为本章收尾。

深度理解

·核心概念：函数式编程（functional programming）是一种"以函数为数据、以组合为手段"的范式。Python 不是纯函数式语言，但提供了足够好用的函数式子集：map/filter/reduce + lambda + 闭包 + 推导式/生成器 + 装饰器。

·底层实现：map/filter 在 Python 3 中返回惰性迭代器（不是列表）——元素在被消费时才逐一产生，这是"延迟计算"思想的体现；reduce 则是一次性把整个迭代压缩（fold）成单个值。

·为什么这样设计：每种范式解决不同的问题——过程式强调"按步骤执行"，OOP 强调"对象与状态"，函数式强调"无副作用的变换管道"。Python 的立场是"多范式融合，按场景取用"。

·实际问题：数据清洗管道（读 → map 转换 → filter 过滤 → reduce 汇总）、Spark 等大数据框架的算子就是 map/filter/reduce 的直接推广。掌握这三件套等于掌握了最通用的"数据处理流水线"词汇。

·初学者误区：① 以为函数式必须"纯函数、零副作用"才能

用——Python 是务实混合体，接受副作用；② 以为三件套只是“语法糖”——它们也是惰性、可组合的抽象，理解这一点才能与生成器章节衔接。

19.6.1 把函数映射到可迭代对象：map (Mapping Functions over Iterables: map)

英文原文

One of the more common things programs do with lists and other sequences is apply an operation to each item and collect the results—selecting columns in database tables, incrementing pay fields of employees in a company, parsing email attachments, and so on. Python has multiple tools that make such collection-wide operations easy to code. For instance, we've seen that updating all the counters in a list can be done easily with a for loop:

```
>>> counters = [1, 2, 3, 4] >>> updated = [] >>> for x in counters: updated.append(x + 10) # Add 10 to each item
```

```
>>> updated, counters ([11, 12, 13, 14], [1, 2, 3, 4])
```

But because this is such a common operation, Python also provides built-ins that do most of the work for you. Among them, the map function applies a passed-in function to each item in an iterable object and returns a list containing all the function-call results. For

example, assuming counters is intact:

```
>>> def inc(x): return x + 10 # Function to be run >
>> list(map(inc, counters)) # Collect call results [11, 12, 13, 14]
```

We met map briefly in Chapters 13 and 14, as a way to apply a built-in function to items in an iterable. Here, we make more general use of it by passing in a user-defined function to be applied to each item in the list—map calls our inc on each list item and collects all the return values into a new list. Remember that map is a nonsequence iterable, so a list call is used to force it to produce all its results for display here (per Chapter 14 coverage). Because map expects a function to be passed in and applied, it also happens to be one of the places where lambda commonly appears:

```
>>> list(map((lambda x: x + 3), counters)) # Function expression [4, 5, 6, 7]
```

Here, the function adds 3 to each item in the counters list; as this little function isn't needed elsewhere, it was written inline as a lambda. Because such uses of map are equivalent to for loops, with a little extra code you can always code a general mapping utility yourself:

```
>>> def mymap(func, iter): res = [] for x in iter: res.append(func(x)) return res
```

Assuming the function inc is still as it was when it was shown previously, we can map it across a sequence (or other iterable) with either

er the built-in or our equivalent:

```
>>> list(map(inc, [1, 2, 3])) # Built-in is an iterable [1, 12, 13]
```

```
>>> mymap(inc, [1, 2, 3]) # Ours builds a list (see generators) [11, 12, 13]
```

However, as `map` is a built-in, it's always available, always works the same way, and may have performance benefits (as we'll prove in Chapter 21, it's faster than a manually coded `for` loop in some usage modes). Moreover, `map` can be used in more advanced ways than shown here. For instance, given multiple sequence arguments, it sends items taken from sequences in parallel as distinct arguments to the function:

```
>>> pow(3, 4) # 34 81
```

```
>>> list(map(pow, [1, 2, 3], [2, 3, 4])) # 12, 23, 34 [1, 8, 81]
```

With multiple sequences, `map` expects an N -argument function for N sequences. Here, the `pow` function takes two arguments on each call—one from each sequence passed to `map`. It's not much extra work to simulate this multiple-sequence generality in code, too, but we'll postpone doing so until later in the next chapter, after we've explored some additional iteration tools (hint: it would be better to generate items on demand, like the built-in). The `map` call is also similar to the list comprehension expressions we studied in C

chapter 14 and will revisit in the next chapter from a functional perspective:

```
>>> list(map(inc, [1, 2, 3, 4])) [11, 12, 13, 14]
```

```
>>> [inc(x) for x in [1, 2, 3, 4]] [11, 12, 13, 14]
```

In some cases, `map` may be faster to run than a list comprehension, and it may also require less code. On the other hand, because `map` applies a function call to each item instead of an arbitrary expression, it is a somewhat less general tool, and often requires extra helper functions or lambdas. Moreover, wrapping a comprehension in parentheses instead of square brackets creates an object that generates values on request to save memory and increase responsiveness, much like `map`—a topic we'll take up in the next chapter.

中文翻译

程序对列表和其他序列做的最常见的事情之一，就是对每一项应用一个操作并收集结果——在数据库表中选择列、给公司里员工的薪资字段加薪、解析邮件附件，等等。Python 有多个工具让这种“全集合操作”易于编写。例如，我们已经看到，用 `for` 循环可以轻松更新列表中的所有计数器：

```
python >>> counters = [1, 2, 3, 4] >>> updated = [] >>> for x in counters: updated.append(x + 10) # 给每项加 10 >>> updated, counters ([11, 12, 13, 14], [1, 2, 3, 4])
```

但正因为这是如此常见的操作，Python 也提供了替你完成大部分工作的内置函数。其中，`map` 函数把一个传入的函数应用到可迭代对象中的每一项，并返回一个包含所

有函数调用结果的列表。例如，假设 counters 仍然完好：

```
python >>> def inc(x): return x + 10 # 要运行的函数>
>> list(map(inc, counters)) # 收集调用结果[11, 12, 13,
14]
```

我们在第 13 章和第 14 章简单见过 map，当时是把它当作“把内置函数应用到可迭代对象各项”的方式。在这里，我们更通用地使用它——传入一个用户自定义函数应用到列表的每一项：map 对每个列表项调用我们的 inc，并把所有返回值收集到一个新列表里。记住 map 是一个非序列的可迭代对象，所以这里用 list 调用强制它产出全部结果以便显示（按第 14 章的讲解）。因为 map 期望传入并应用一个函数，它恰好也是 lambda 常见出没的地方之一：

```
python >>> list(map((lambda x: x + 3), counters)) # 函数
表达式[4, 5, 6, 7]
```

这里，这个函数给 counters 列表的每一项加 3；由于这个小函数在别处用不到，它被写成内联的 lambda。因为这样使用 map 等价于 for 循环，再多写一点代码，你总是可以自己编写一个通用的映射工具：

```
python >>> def mymap(func, iter): res = [] for x in iter: res.a
ppend(func(x)) return res
```

假设 inc 函数还像前面展示的那样，我们可以用内置的 map 或我们自己的等价实现把它映射到整个序列（或其他可迭代对象）上：

```
python >>> li
st(map(inc, [1, 2, 3])) # 内置版本是一个可迭代对象[11, 1
2, 13]
```

```
>>> mymap(inc, [1, 2, 3]) # 我们的版本构建列表
（参见生成器）[11, 12, 13]
```

然而，由于 map 是内置函数，它始终可用、行为始终一致，而且可能具有性能优势（我们将在第 21 章证明，在某些使用模式下它比手工编写的 for 循环更快）。此外，map 的使用方式可以比这里展示的更进一步。例如，给定多个序列参数，它会把从各序列并行取出的项作为独立参数传给函数：

```
python >>> pow(3, 4)
```



```
# 34 81 >>> list(map(pow, [1, 2, 3], [2, 3, 4])) # 12, 23, 34
```

[1, 8, 81] 使用多个序列时，map 期望 N 个序列对应一个 N 参数函数。这里，pow 函数在每次调用时接收两个参数——一个来自传给 map 的每个序列。在代码中模拟这种多序列通用性也不太费事，但我们要把它推迟到下一章晚些时候，等我们探索了一些额外的迭代工具之后（提示：像内置版本那样按需生成（generate）项会更好）。map 调用也与我们在第 14 章学习的列表推导式相似，下一章会从函数式视角重新审视它们：python >>> list(map(inc, [1, 2, 3, 4])) [11, 12, 13, 14] >>> [inc(x) for x in [1, 2, 3, 4]] [11, 12, 13, 14] 在某些情况下，map 的运行速度可能比列表推导式更快，而且需要的代码也可能更少。另一方面，因为 map 对每项应用的是函数调用而不是任意表达式，它是一个通用性略低的工具，常常需要额外的辅助函数或 lambda。此外，用圆括号而不是方括号包住推导式，会创建一个按需生成值的对象，以节省内存并提高响应速度——很像 map——这是下一章要讨论的主题。

深度理解

·核心概念：map(func, iterable) = "把函数批量应用到每个元素"。它是循环 + 收集结果这个惯用模式的"一站式"封装。等价的推导式是 [func(x) for x in it]。

·底层实现：Python 3 中 map 返回 map 对象（惰性迭代器）：调用 map(inc, counters) 时没有执行任何 inc，只是创建了迭代器；每次 next()（或 list() 强制迭代）才取一个元素、调一次 inc。多序列时它内部用 zip 式的并行取数（最短序列耗尽即停），把每组元素作为位置参数传

给函数——这就是 `map(pow, [1,2,3], [2,3,4])` 得到 `[1, 8, 81]` 的机制。

·为什么这样设计：惰性 = "按需计算"——处理 10 亿行数据时，不用先构建 10 亿元素的中间列表，取多少算多少。作者反复埋下"generators"的伏笔：`map` 本身就是一个"没有 `yield` 的生成器"。性能上，C 实现的 `map` 循环比 Python 层 `for` 循环更快（第 21 章实测）。

·实际问题：批量转换（字符串去空格、金额换算、状态编码）、并行多序列对齐计算（`map(divmod, nums, mods)`）、把函数批量应用到数据库查询结果集。但代码可读性上推导式通常更优——`[x+3 for x in counters]` 比 `list(map(lambda x: x+3, counters))` 更直白。

·初学者误区：① 忘了包 `list()`，直接 `print(map(inc, counters))` 看到 `<map object at 0x...>` 以为出 bug——它是惰性迭代器，正常现象；② 以为 `map` 的多个序列是"组合"（笛卡尔积）——不是，是并行对齐（`zip` 语义）；③ 给 `lambda` 加无谓的括号 `list(map((lambda x: x+3), counters))`——括号无害但多余，作者保留是为了清晰。

·设计思想：`map` 与推导式的"同构"（isomorphic）是本章的重要伏笔：下一章将证明生成器表达式 $\approx$ 惰性 `map` + 惰性 `filter` 的组合。先记住"map 压上了惰性"这个印象，后面自然水到渠成。

19.6.2 在可迭代对象中筛选元素：filter (Selecting Items in Iterables: filter)

英文原文

The map function is a primary and relatively straightforward representative of Python's functional programming toolset. Its close relatives, filter and reduce, select an iterable's items based on a test function and apply functions to item pairs, respectively.

Because it also returns an iterable, filter (like range) requires a list call to display all its results in a REPL. For example, the following filter call picks out items in a sequence that are greater than zero:

```
>> list(range(-5, 5)) [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4]
>> list(filter((lambda x: x > 0), range(-5, 5))) [1, 2, 3, 4]
```

We met filter briefly earlier in the sidebar "Why You Will Care: Booleans" and while exploring iterables in Chapter 14. Items in the sequence or iterable for which the function returns a true result are added to the result list. Like map, this function is roughly equivalent to a for loop, but it is built-in, concise, and often faster:

```
>> res = []
>> for x in range(-5, 5):
    if x > 0: # The statement equivalent of filter
        res.append(x) # Simple but slower today, probably
>> res [1, 2, 3, 4]
```

Also like map, filter can be emulated by list comprehension syntax with often simpler results (especially w

hen it can avoid creating a new function), and with a similar generator expression when coded with enclosing parentheses to delay production of results—though again, the generator story will have to remain a teaser for the next chapter:

```
>> [x for x in range(-5, 5) if x > 0] [1, 2, 3, 4]
```

中文翻译

map 函数是 Python 函数式编程工具箱中一个主要且相对直白的代表。它的近亲 filter 和 reduce，分别基于测试函数筛选可迭代对象的项、以及把函数应用于项对。由于它也返回一个可迭代对象，filter（像 range 一样）需要一个 list 调用才能在 REPL 中显示其全部结果。例如，下面的 filter 调用挑出序列中大于零的项：python >>> list(range(-5, 5)) [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4] >>> list(filter((lambda x: x > 0), range(-5, 5))) [1, 2, 3, 4] 我们之前在侧边栏《你会在意的原因：布尔值》以及第 14 章探索可迭代对象时，简要见过 filter。序列或可迭代对象中、能让该函数返回真值（true）结果的项，会被加入结果列表。和 map 一样，这个函数大致等价于一个 for 循环，但它是内置的、简洁的，而且往往很快：python >>> res = [] >>> for x in range(-5, 5): if x > 0: # 与 filter 等价的语句形式 res.append(x) # 简单，但今天可能更慢 >>> res [1, 2, 3, 4] 与 map 一样，filter 也可以用列表推导式语法模拟，结果往往更简单（尤其是当它可以避免创建新函数时）；用包围的圆括号写成生成器表达式（generator expression）时效果类似，会延迟结果的生产——不过再说一次，生

成器的故事只能留作下一章的预告：`python >>> [x for x in range(-5, 5) if x > 0] [1, 2, 3, 4]`

深度理解

- 核心概念：`filter(func, iterable)` = "按测试函数筛选"。保留条件是"函数返回真值"——注意不是 `True` 字面量，而是"真值语义"：`0`、`''`、`[]`、`None` 都会被滤掉。两个参数都可省略 `func` 的情况 (`filter(None, it)`) 会滤掉所有假值项。
- 底层实现：`filter` 同样是惰性迭代器 (`filter object`)。每次 `next()` 从底层可迭代对象取一个元素，调用测试函数：真则产出该元素，假则跳过继续取下一个，直到耗尽。对比 `map` (对每项必调用) 与 `filter` (可能跳过不调用)——`map` 保证输出长度 = 输入长度，`filter` 输出 $\leq$ 输入。
- 为什么这样设计：与循环版对比，`filter` 的优点是"意图显式化"——`filter(pred, data)` 一眼可见"筛选"，而循环+`if`+`append` 需要读者解读三行代码。作者如实说明循环版"今天可能更慢"——内置函数在 C 层实现，避免 Python 层每元素一次解释执行。
- 实际问题：日志过滤 (保留 `ERROR` 级)、数据清洗 (剔除空值/非法值)、权限筛选 (保留有权限的用户)。工程中更常见的替代是推导式的 `if` 子句 `[x for x in data if x > 0]`——作者承认推导式"结果往往更简单"。
- 初学者误区：① 忘了 `list()` 包裹；② 把测试函数写成"返回 `bool` 字符串"之类——函数按真值判定，写 `x > 0` 或 `lambda x: x % 2` (奇数) 都对，但返回非 `bool` 也不会报

错，可能误导；③ 认为 filter 必须传函数——filter(None, seq) 去掉所有假值项是实用技巧，很多老手才知道。

·设计思想：注意三种写法的分工：filter（函数式）、推导式（声明式）、for 循环（命令式）——同一功能三种范式，正是本章标题“工具杂锦”的缩影。Python 鼓励“按场合挑选”，但可读性优先。

19.6.3 合并可迭代对象中的项：reduce (Combining Items in Iterables: reduce)

英文原文

The functional reduce call—once a built-in but now a resident of the functools standard-library module—is more complex. It accepts an iterable to process, but it's not an iterable itself: it returns a single result that aggregates an iterable's items. To demo, here are two reduce calls that compute the sum and product of the items in a list:

```
>> from functools import reduce >> reduce((lambda  
a x, y: x + y), [1, 2, 3, 4]) 10 >> reduce((lambda x, y: x  
y), [1, 2, 3, 4]) 24
```

At each step, reduce passes the current sum or product, along with the next item from the list, to the passed-in lambda function. By default, the first item in the sequence initializes the starting value. To make that more concrete again, here's the for loop equivalent

t to the first of these calls, with the addition hardcoded inside the loop:

```
>> L = [1, 2, 3, 4] >> res = L[0] >> for x in L[1:]: res  
+= x >> res  
10
```

In fact, coding your own reusable and customizable version of `reduce` is fairly straightforward too. The following function emulates most of the built-in's behavior and helps demystify its operation in general:

```
>> def myreduce(function, sequence):  
tally = sequence[0] for next in sequence[1:]: tally = fu  
nction(tally, next) return tally >> myreduce((lambda  
x, y: x + y), [1, 2, 3, 4])  
10 >> myreduce((lambda x, y:  
x * y), [1, 2, 3, 4])  
24
```

The built-in `reduce` also allows an optional third argument, effectively placed before the items in the sequence to serve as an initial value and a default result when the sequence is empty, but we'll leave this extension as a suggested exercise (again, emulating built-in tools is instructive, but superfluous).

If this coding technique has sparked your interest, you might also be interested in the standard-library `operator` module, which provides functions that correspond to built-in expressions and so comes in handy for some uses of functional tools (consult help in a REPL or Python's library manual for more details on this module):

```
>> import operator, functools >> functools.reduce(  
operator.add, [2, 4, 6]) # Function-based + 12 >> fun  
ctools.reduce((lambda x, y: x + y), [2, 4, 6]) 12
```

Together, map, filter, and reduce support powerful functional programming techniques. As mentioned, many observers would also extend the functional programming toolset in Python to include the nested function closures and anonymous function lambdas we've explored, as well as the generators and comprehensions we've met in piecemeal fashion along the way. To fully grok the latter two, though, we must move on to the next chapter.

中文翻译

函数式的 reduce 调用——曾经是内置函数，如今是标准库 functools 模块的居民——更为复杂。它接收一个要处理的可迭代对象，但它本身不是可迭代对象：它返回一个聚合可迭代对象各项目的单一结果。演示一下，下面是两次计算列表中各项之和与之积的 reduce 调用：python >>> from functools import reduce >>> reduce((lambda x, y: x + y), [1, 2, 3, 4]) 10 >>> reduce((lambda x, y: x * y), [1, 2, 3, 4]) 24 在每一步，reduce 都把当前的和或积，连同列表中的下一个项，传给传入的 lambda 函数。默认情况下，序列中的第一项初始化起始值。为了让这一点更具体，下面是第一次调用等价的 for 循环，加法被硬编码在循环内：python >>> L = [1, 2, 3, 4] >>> res = L[0] >>> for x in L[1:]: res += x >>> res 10 事实上，编写自己的、可复用可定制的 reduce 版本也相当直白。下面这个函数模拟了

内置版本的大部分行为，有助于揭开它的神秘面纱：`python >>> def myreduce(function, sequence): tally = sequence[0] for next in sequence[1:]: tally = function(tally, next) return tally >>> myreduce((lambda x, y: x + y), [1, 2, 3, 4]) 10 >>> myreduce((lambda x, y: x y), [1, 2, 3, 4]) 24` 内置的 `reduce` 还允许一个可选的第三个参数，它实际上被放在序列各项之前，充当初始值，并在序列为空时作为默认结果——不过我们把这个扩展留作建议练习

（再说一次，模拟内置工具是有教育意义的，但也多此一举）。如果这种编码技巧激起了你的兴趣，你可能也会对标准库的 `operator` 模块感兴趣，它提供了与内置表达式对应的函数，因此对函数式工具的一些使用很方便（关于这个模块的更多细节，请在 REPL 中查看 `help` 或查阅 Python 库手册）：`python >>> import operator, functools >>> functools.reduce(operator.add, [2, 4, 6]) # 基于函数的 + 12 >>> functools.reduce((lambda x, y: x + y), [2, 4, 6]) 12` 合在一起，`map`、`filter` 和 `reduce` 支撑起了强大的函数式编程技术。如前所述，许多观察者还会把嵌套函数闭包、我们已经探索过的匿名函数 `lambda`，以及一路上零散遇到的生成器和推导式，都算进 Python 的函数式编程工具箱。不过要真正理解后两者（`generators` 和 `comprehensions`），我们必须进入下一章。

深度理解

·核心概念：`reduce(func, iterable) = "把可迭代对象折叠（fold）成单个值"`。二元函数依次作用于"累计值 + 下一项"： $((1+2)+3)+4 = 10$ 。它与 `map/filter` 的根本区别：输出不是序列，而是标量。

·底层实现：reduce 是立即执行的（非惰性——它必须遍历完整个可迭代对象才能返回单个结果）。内部逻辑就是 `tally = seq[0]`，然后循环 `tally = func(tally, x)`。注意 `next` 在示例里用作变量名——这遮蔽了内置 `next` 函数（作者示例本身有此瑕疵，可读性稍差，但功能正确）。历史：reduce 在 Python 2 是内置函数，Python 3 中被移入 `functools`——Guido 认为“可读性差、滥用多”，这是 Python 语言史上一次著名的“降级”。

·为什么这样设计：为什么需要 reduce？——求和、求积、求最大值、连接字符串、计算平均、解析嵌套结构，都是“折叠”。有 `sum()` 为何还要 reduce？——`sum` 只处理加法，reduce 接受任意二元函数，是通用折叠。但通用性也带来滥用风险——`functools` 的地位变化是“可用但非首选”的官方态度。

·实际问题：`operator` 模块把 `+`、`*`、`>` 等运算符变成函数（`operator.add`、`operator.mul`、`operator.itemgetter`），避免每次写 `lambda`。`functools.reduce(operator.mul, nums, 1)` 是求积的惯用法。工程上更常见的是 `max`/`min`/`sum`/`join` 等专用折叠函数——它们是 reduce 的“特化版本”，更快更可读。

·初学者误区：① 忘了 `import`——reduce 不在内置命名空间（Python 3），直接调用报 `NameError`；② 传给 reduce 的函数必须二元（两个参数）——一元的会报 `Type Error`；③ 空序列无初始值会抛 `TypeError`（没有可作起点的项）——第三个参数正是为此设计的；④ 把 `operator.add` 当字符串或忘了传——它是个函数对象，直接传

名，不要加括号。

·设计思想：作者自己都写了 `myreduce` 模拟——“模拟内置工具有教育意义的，但也多此一举”。这是他一贯的教学手法：先拆开看原理，再用内置。同时为下一章铺路：理解“折叠”后，生成器表达式与 `reduce` 的组合（惰性 + 折叠）就顺理成章了。

19.7 章节小结 (Chapter Summary)

英文原文

This chapter's collage took us on a tour of function-related topics: function design guidelines; recursive functions; function attributes, annotations, and decorators; lambda expressions; and the `map`, `filter`, and `reduce` built-ins. Some of these are advanced, but most are common in Python programming. The next chapter continues the advanced-topics motif with a reprisal of comprehensions and an unmasking of generators—tools that are just as related to functional programming as to looping statements. Before you move on, though, make sure you've mastered the concepts covered here by working through this chapter's quiz.

中文翻译

本章的拼贴画带我们巡游了与函数相关的主题：函数设计准则；递归函数；函数属性、注解和装饰器；lambda 表达式；以及 `map`、`filter` 和 `reduce` 内置函数。其中有些是高级

内容，但大多数在 Python 编程中都很常见。下一章继续"高级主题"的主线，将重新审视推导式并揭开生成器的面纱——这些工具与函数式编程的关系和与循环语句的关系一样密切。不过在你继续之前，请务必通过本章的测验来确认自己已掌握这里的概念。

19.8 测验：检验你的知识 (Test Your Knowledge: Quiz)

英文原文

1. How are lambda expressions and def statements related? 2. What's the point of using lambda? 3. Compare and contrast map, filter, and reduce. 4. What are function annotations, and how are they used? 5. What are recursive functions, and how are they used? 6. What are some general design guidelines for coding functions? 7. Name three or more ways that functions can communicate results to a caller.

中文翻译

1. lambda 表达式和 def 语句有什么关系？2. 使用 lambda 的意义是什么？3. 比较并对比 map、filter 和 reduce。4. 函数注解是什么，它们如何使用？5. 递归函数是什么，它们如何使用？6. 编写函数有哪些通用设计准则？7. 说出至少三种函数向调用者传达结果的方式。

19.9 测验答案 (Test Your Knowledge: Answers)

英文原文

1. Both lambda and def create function objects to be called later. Because lambda is an expression, though, it returns a function object instead of assigning it to a name, and it can be used to nest a function definition in places where a def will not work syntactically. A lambda allows for only a single implicit return value expression, though; because it does not support a block of statements, it is not ideal for larger functions. 2. lambda allows us to "inline" small units of executable code, defer its execution, and provide it with state in the form of default arguments and enclosing scope variables. Using a lambda is never required; you can always code a def instead and reference the function by name. lambda comes in handy, though, to embed small pieces of deferred code that are unlikely to be used elsewhere in a program. It commonly appears in callback-based programs such as GUIs, and has a natural affinity with functional tools like map and filter that expect a processing function. 3. These three built-in functions all apply another function to items in a sequence (or other iterable) object and collect results. map passes each item to the function and collects all results, filter collects items for which the function

returns a true value, and reduce computes a single value by applying the function to an accumulator and successive items. Unlike the other two, reduce is available in the functools module, not the built-in scope (in modern Python history, at least).

4. Function annotations are syntactic embellishments of a function's arguments and result, which are collected into a dictionary assigned to the function's annotations attribute. Python places no semantic meaning on these annotations, but simply packages them for potential use by other tools.
5. Recursive functions call themselves either directly or indirectly in order to loop (i.e., repeat). They may be used to traverse arbitrarily shaped structures, but they can also be used for iteration in general (though the latter role is often more simply and efficiently coded with looping statements). Recursion can often be simulated or replaced by code that uses explicit stacks or queues to have more control over traversals.
6. Functions should generally be small and as self-contained as possible, have a single unified purpose, and communicate with other components through input arguments and return values. They may use mutable arguments to communicate results too if changes are expected, and some types of programs imply other communication mechanisms.
7. Functions can send back results with return statements, by changing passed-in mutable arguments, and by setting global variables. Globals are generally frowned upon (e

xcept for very special cases, like multithreaded programs) because they can make code more difficult to understand and use. return statements are usually best, but changing mutables is fine (and even useful), if expected. Functions may also communicate results with system devices such as files and sockets, but these are beyond our scope here.

中文翻译

1. lambda 和 def 都创建供稍后调用的函数对象。不过，因为 lambda 是表达式，它返回函数对象而不是把它赋值给名字，而且它可以用来在 def 语法上无法工作的地方嵌套函数定义。不过 lambda 只允许一个单一的隐式返回表达式；因为它不支持语句块，所以不适合较大的函数。2. lambda 让我们能够“内联”小块可执行代码、延迟其执行，并通过默认参数和包围作用域变量的形式为它提供状态。使用 lambda 从来不是必需的；你总是可以改用 def 并按名字引用函数。不过，要嵌入程序中其他地方不太可能用到的小块延迟代码时，lambda 很方便。它常出现在 GUI 等基于回调的程序中，并且与 map、filter 这类期望处理函数的函数式工具有天然的亲和力。3. 这三个内置函数都把另一个函数应用到序列（或其他可迭代对象）对象中的项上并收集结果。map 把每个项传给函数并收集调用结果，filter 收集函数返回真值的项，reduce 通过把函数应用到累计器（accumulator）和连续的项上来计算单个值。与另外两个不同，reduce 位于 functools 模块中，而不是内置作用域（至少在近代 Python 历史中如此）。4. 函数注解是函数参数和结果的语法装饰，会被收集进一个字典并赋给函数的 annot

ations 属性。Python 对这些注解不赋予任何语义含义，只是把它们打包起来，供其他工具可能使用。

5. 递归函数直接或间接调用自身以构成循环（即重复）。它们可以用来遍历任意形状的结构，也可以用于一般性的迭代（不过后一种角色通常用循环语句编码更简单高效）。递归常常可以被使用显式栈或队列的代码模拟或替代，以获得对遍历的更多控制。

6. 函数通常应该小而尽量自包含，有单一统一的目的，并通过输入参数和返回值与其他组件通信。如果变更被期望，它们也可以使用可变参数传达结果，某些类型的程序还暗示着其他通信机制。

7. 函数可以用 `return` 语句、修改传入的可变参数、以及设置全局变量来传回结果。全局变量通常不受待见（除了一些非常特殊的场景，比如多线程程序），因为它们会让代码更难理解和使用。`return` 语句通常是最好的，但如果变更符合预期，修改可变对象也没问题（甚至有用）。函数还可以通过文件和套接字等系统设备传达结果，但这些超出了本书范围。

19.10 侧边栏：你会在意的原因——lambda 回调 (Why You Will Care: lambda Callbacks)

英文原文

Another common role for lambda is to define inline callback functions for Python's tkinter GUI API. For example, the following creates a button that prints a message on the console when pressed, assuming tkinter is present in your Python (it is by default on most PC

s and at least one Android app):

```
from tkinter import Button, mainloop
x = Button( text='Press me', command=lambda: print('Tapped!') )
x.pack()
mainloop() # This may be optional in some REPLs
```

Here, we register the callback handler by passing a function generated with a lambda to the `command` keyword argument. The advantage of lambda over `def` in this is that the code that handles a button press is right here, embedded in the button-creation call. In effect, the lambda defers execution of the handler until the event occurs: the `print` call happens on button presses, not when the button is created, and effectively "knows" the string it should write when the event occurs. Real GUIs rarely print to consoles, of course, but this demos the coding pattern. Because the nested function scope rules apply to lambda, they automatically see names in the functions in which they are coded and hence don't require passed-in defaults in most cases (except for loop variables, per Chapter 17). This is especially useful for accessing the special `self` instance argument that is a local variable in enclosing class method functions (which we'll study in Part VI, so take this as preview only):

```
class MyGui:
    def makewidgets(self):
        Button(command=lambda: self.onPress('Tapped!'))
```

```
def onPress(self, message):...use message...
```

As you'll see later, both class objects with call and bound methods often serve in callback roles too—watch for coverage of these in Chapters 30 and 31. And all of this applies to coding event callbacks in other commonly used and portable GUI toolkits for Python—including Kivy, Toga (in BeeWare), PyQt, and wxPython. tkinter gets more press here only because it's shipped with Python's standard library. As for all tools, you should vet GUIs for yourself.

中文翻译

lambda 的另一个常见角色是为 Python 的 tkinter GUI API 定义内联回调函数。例如，下面这段代码创建了一个按钮，按下时在控制台打印一条消息——假设 tkinter 存在于你的 Python 中（在大多数 PC 上默认如此，至少在一款 Android 应用上也是）：

```
python from tkinter import Button, mainloop  
x = Button( text='Press me', command=  
=lambda: print('Tapped!') )  
x.pack()  
mainloop()
```


在某些 REPL 中这一步可能是可选的
在这里，我们通过把一个 lambda 生成的函数传给 command 关键字参数来注册回调处理器。此场景下 lambda 相对 def 的优势是：处理按钮按下的代码就在这里，嵌入在按钮创建调用之中。实际上，lambda 延迟了处理器的执行，直到事件发生：print 调用发生在按钮被按下时，而不是按钮被创建时，并且它有效地“知道”事件发生时应该写入的字符串。真实的 GUI 当然很少向控制台打印，但这演示了编码模式。因为嵌套函数作用域规则同样适用于 lambda，它们会自动看到定义它们的函数。

数中的名字，因此大多数情况下不需要传入的默认值（循环变量除外，参见第 17 章）。这一点对访问特殊的 self 实例参数尤其有用——self 是包围它的类方法函数中的局部变量（我们将在第六部分学习，所以这里只是预告）：`python class MyGui: def makewidgets(self): Button(command=lambda: self.onPress('Tapped!')) def onPress(self, message):...使用 message...你稍后会看到，带有 call 的类对象和绑定方法（bound methods）也常常扮演回调角色——请关注第 30 章和第 31 章的介绍。而这一切同样适用于为其他常用且可移植的 Python GUI 工具包编写事件回调——包括 Kivy、Toga（在 BeeWare 中）、PyQT 和 wxPython。tkinter 在这里得到更多笔墨，仅仅是因为它随 Python 标准库一同发布。至于所有工具，你应该亲自评估各种 GUI。`

深度理解

- 核心概念：回调（callback） = "把函数交给框架，让框架在事件发生时调用它"。GUI 是回调的经典舞台：你不主动调用按钮的处理器，按钮被按下时框架替你调用。
- 底层实现：`Button(command=lambda: print('Tapped!'))` 把 lambda 函数对象存入按钮对象内部。事件循环（`mainloop()`）持续监听鼠标/键盘事件；检测到点击时，Tkinter 的 C 层回调机制调用 `command` 里存的函数。关键：创建按钮时 `print` 不会执行——lambda 只是被"存放"，`print` 在事件发生时执行——这就是"延迟执行"（deferring execution）的机制。
- 为什么这样设计：为什么不用 `def`？——可以，但 `def` 必

须定义在文件某处再按名引用，处理逻辑与按钮创建被拆散；lambda 让"发生了什么"和"在哪处理"同框出现（代码邻近性）。闭包规则让 lambda 能访问外层 self——`command=lambda: self.onPress('Tapped!')` 中的 self 来自 `makewidgets` 方法的作用域，调用时依然有效（LEGB 的 E 在起作用）。

·实际问题：所有现代 UI 框架（Tkinter、PyQt、wxPython、Kivy）与前端 JavaScript 的事件模型都是回调。经典陷阱（第 17 章预告的）：循环里创建多个按钮，回调捕获的是循环变量最终值——因为闭包捕获的是变量本身而非当时的值。解法是把值做成默认参数 `command=lambda i=i: ...`。

·初学者误区：① 误写 `command=print('Tapped!')`（带括号）——这会在创建按钮时就执行打印，且传入的是返回值 `None`，点击毫无反应——正是"延迟执行"的反面教材；② 以为回调在注册时运行——不，注册只是"登记"；③ 忘了 `mainloop()` 程序就立即退出——事件循环是 GUI 的"心跳"。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

·递归：目录树遍历、JSON/XML 解析、组织架构图递归

汇总、网页链接图爬取（配合 visited 集合防环）、排列组合与回溯算法（八皇后、数独求解）。

· lambda: GUI 回调 (command=lambda: ...)、sorted/min/max 的 key 参数 (key=lambda item: item['price'])、跳转表/分发表 (CLI 子命令映射、状态机迁移表)。

· 函数属性: 注册框架元数据 (如把路由路径挂在函数上)、调试用调用计数、把状态随函数一起打包传递。

· 注解: 类型提示 (mypy/Pyright 静态检查)、API 契约描述、参数校验工具 (inspect + annotations 实现运行时校验, 第 39 章主题)。

· 装饰器: 日志、计时、缓存 (functools.lru_cache)、重试、权限校验、@staticmethod/@property——“横切关注点”的统一挂载点。

· map/filter/reduce: 数据清洗管道 (转换→过滤→聚合)、多序列并行计算、operator 模块配合的函数式简写。

与其他语言的区别

特性	Python	Java/C++	说明
一等函数	完全支持: 函数可赋值、传参、存储、返回	Java 8 前不支持 (需函数对象/接口); C++ 需函数指针/std::function	一等函数是函数式编程的前提
lambda 语法	lambda x: x+1 (单表达式, 无语句)	Java (x) -> x+1; C++ &{ return x+1; } (支持语句块)	Python 刻意限制 lambda 体为表达式, 保可读性

递归深度限制	默认 1000 层, sys.setrecursionlimit 可调	栈深度由线程栈大小决定 (通常更深, 可设置)	Python 递归更"脆弱", 倾向用显式栈替代
函数注解	def f(a: int) -> bool, 运行时零强制	Java 类型即编译期强制; C++ 无此概念 (用模	Python 注解是"软元数据", 靠外部工具
reduce 地位	从内置降级到 functools	Java Stream .reduce() 是主流 API	Python 对 reduce 的态度: "可用但谨慎"
惰性函数式工具	map/filter 返回惰性迭代器	Java Stream 惰性; C++ range_s 惰性 (C++20	现代语言普遍采纳"按需计算"

+))

Python 发展历史背景

· reduce 的降级: Python 3.0 (2008 年) 中, Guido van Rossum 以"可读性差、易被滥用"为由把 reduce 从内置函数移到 functools 模块——它是"Python 3 大改造" (print 变函数、整数除法语义变更等) 中著名的取舍之一, 也反映了 Python 对可读性的极致偏执。

· lambda 的历史: Python 的 lambda 源自 Lisp (1960 年代), Lisp 从 λ 演算 (1930 年代, Alonzo Church) 继承"匿名函数"概念。Python 的 lambda 刻意比 Lisp/Lisp 系语言更受限——只允许表达式, 是 Python 哲学"宁简勿繁"的体现。

· 注解与类型提示: PEP 3107 (函数注解, 2006) 定义语法但明确"不做语义"; PEP 484 (类型提示, 2014) 把注解的"类型角色"发扬光大; PEP 563 (延迟求值注解) 则是注解求值策略的演进。今天注解生态已形成 (mypy、pyright、pydantic), 但作者提醒其角色仍有争议。

· 装饰器: PEP 318 (2004) 引入 @ 语法, 最初灵感来自

Java 注解与 AOP（面向切面编程）思想，如今是 Python 最成功的高级特性之一。

高级开发者需要掌握的相关知识

- inspect 模块：用 inspect.signature(func) 获取参数信息，比直接读 code 更上层。
- functools 全套：partial（柯里化工具）、lru_cache（记忆化）、wraps（保留函数元数据的装饰器工具）、singledispatch（泛型分发）。
- collections.deque：O(1) 两端操作，替代示例中 pop(0) 的低效队列写法。
- 迭代器协议与生成器：iter/next、yield——下一章的核心，map/filter 的惰性本质将在此彻底展开。
- 装饰器链与带参装饰器：@a @b def f 的展开顺序（自下而上应用）。
- 深递归的工程替代：把递归改写成显式栈/队列，绕开调用栈深度限制（如遍历超大 JSON）。

学习建议 (Learning Advice)

重要程度

★★★★★ (5/5 星) ——本章是函数主题的"毕业考试"：递归、一等函数、lambda、map/filter/reduce 在任何实际项目中都会遇到。尤其 lambda 与 map/filter 的组合、函数属性与注解，是阅读第三方框架源码的必备词汇。装饰器虽只预告，但它是理解现代 Python 框架 (Flask

、pytest) 的门票。

应该掌握到什么程度

·必须熟练：lambda 与 def 的区别（表达式 vs 语句）及何时用谁；list(map(...))/list(filter(...)) 的惰性语义与 list() 包裹；from functools import reduce 及 reduce 的折叠过程；递归的“递推 + 基准”模板；函数即对象（赋值、传参、返回）。

·理解原理：函数属性（dict）、注解（annotations 字典）、内省（code.covarnames）、装饰器语法糖的展开形式（func = decorator(func)）。

·能看懂即可：用表达式模拟语句的各种 lambda 技巧（知道“能但别用”）；嵌套 lambda；显式队列/栈遍历的两种变体。

·动手实验：亲手运行 mysum、sumtree 三个版本并对照输出顺序（BFS vs DFS）；把 print 加进递归函数观察层级；写一个带注解的 def 打印 annotations。

后续应该学习哪些相关内容

·第 20 章（立即）：生成器函数与生成器表达式、推导式的函数式视角——map/filter 的惰性本质在此揭晓。

·第 21 章：定时器案例——用实测数据回答“map 比循环快吗？”。

·第 32、39 章：装饰器完整实现（带参装饰器、装饰器类、用注解做参数校验）。

- 第六部分（30~31 章）：类的 call 与绑定方法——回调角色的另一种形态；self 与闭包的结合。
- 标准库自修：functools、operator、inspect、collections.deque，配合 help() 探索。